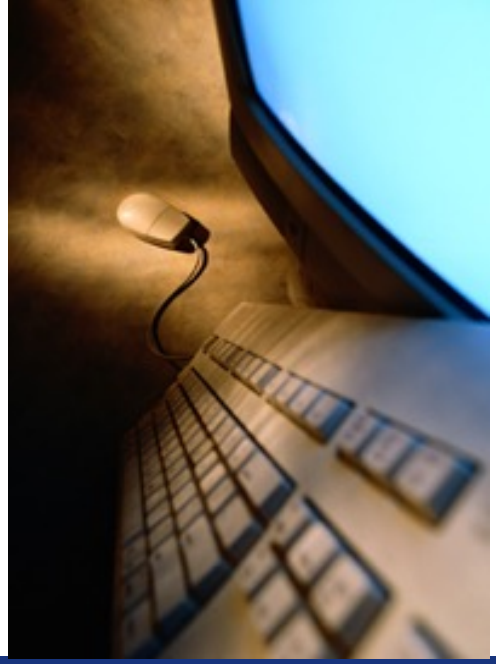
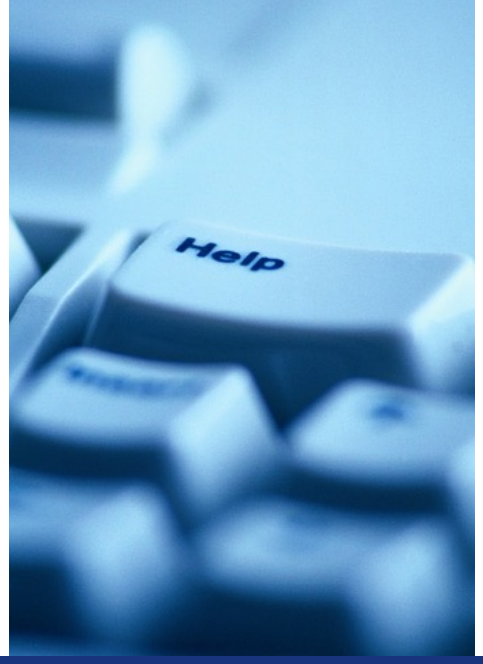
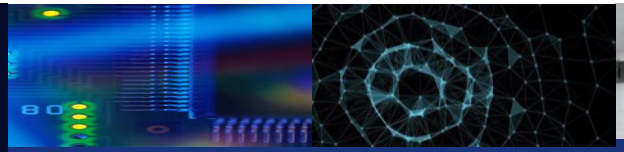



操作系统原理

Operating Systems Principles

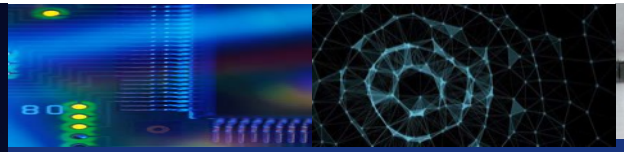
陈鹏飞
计算机学院

第四讲 —进程



概述

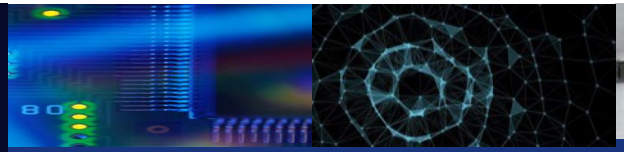
- 进程概念
- 进程调度
- 进程上的操作
- 进程间通信
- IPC在共享存储系统中的应用
- 消息传递系统中的IPC
- IPC系统示例
- 客户机-服务器系统中的通信



进程终止

- 进程执行最后一条语句，然后调用exit（）系统调用请求操作系统删除它。
  - 将状态数据从子进程返回到父进程（通过wait（））
  - 进程的资源由操作系统释放
- 父进程可以使用abort（）系统调用终止子进程的执行。这样做的一些原因：
  - 子进程已超出分配的资源
  - 不再需要分配给子进程的任务
  - 父进程正在退出，如果父进程终止，操作系统不允许子进程继续

Demo

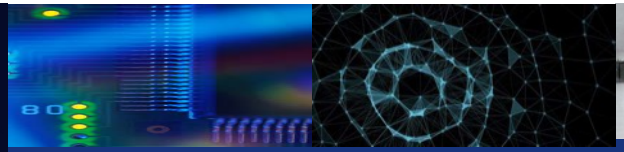


进程终止

- 如果其父进程已终止，则某些操作系统不允许子进程存在。如果进程终止，则其所有子进程也必须终止。
  - 级联终止。所有子女、孙辈等均被终止。
  - 终止由操作系统启动。
- 父进程可以使用wait（）系统调用等待子进程的终止。该调用返回终止进程的状态信息和pid

pid=等待（&状态）；

- 若并没有父进程等待（并没有调用wait（））， 那个么进程就是一个僵尸
- 若父进程在未调用wait（）的情况下终止，则该进程是孤立进程

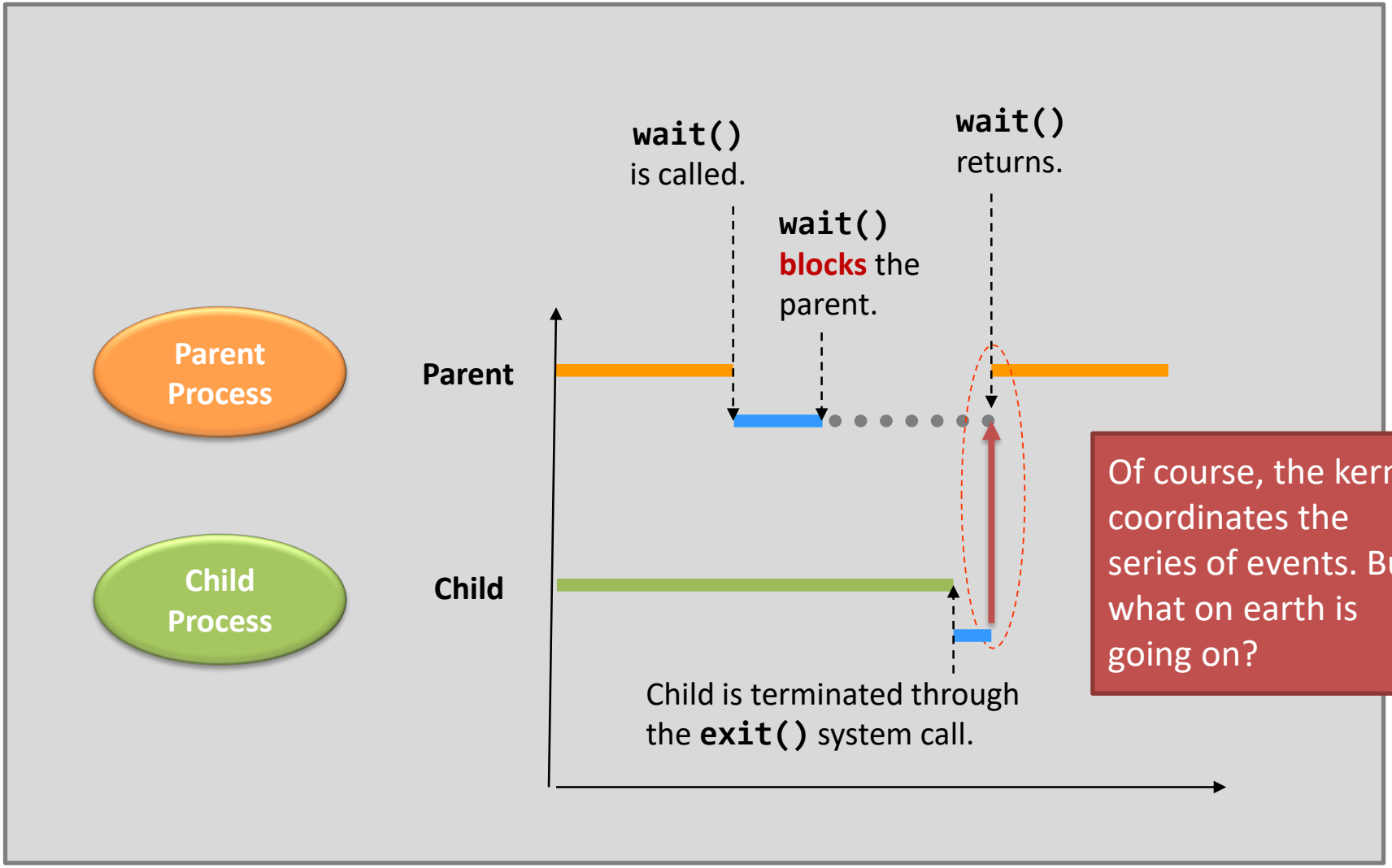


进程终止

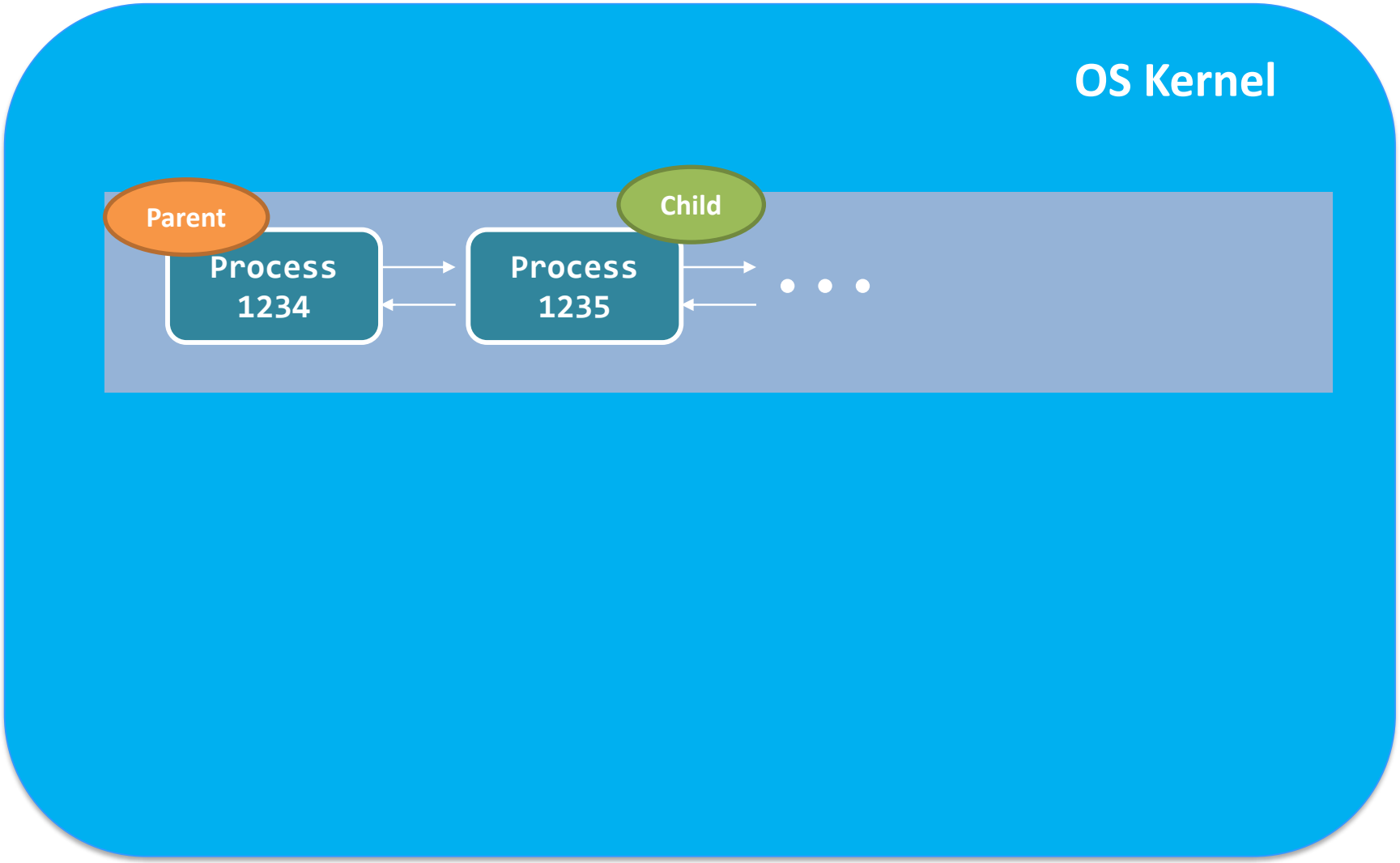
Demo

```
int main(void)
{
    int count = 1;
    pid_t childpid, terminatedid;

    childpid = fork(); /* child duplicates parent's address space */
    if (childpid < 0) {
        perror("fork()");
        return EXIT_FAILURE;
    }
    else
        if (childpid == 0) { /* This is child pro */
            count++;
            printf("child pro pid = %d, count = %d (addr = %p)\n", getpid(), count,
&count);
            printf("child sleeping ...\n");
            sleep(5); /* parent wait() during this period */
            printf("\nchild waking up!\n");
        }
        else { /* This is parent pro */
            terminatedid = wait(0);
            printf("parent pro pid = %d, terminated pid = %d, count = %d (addr = %p)\n",
getpid(), terminatedid, count, &count);
        }
    printf("\nTesting point by %d\n", getpid()); /* executed by child and parent */
    return EXIT_SUCCESS;
}
```

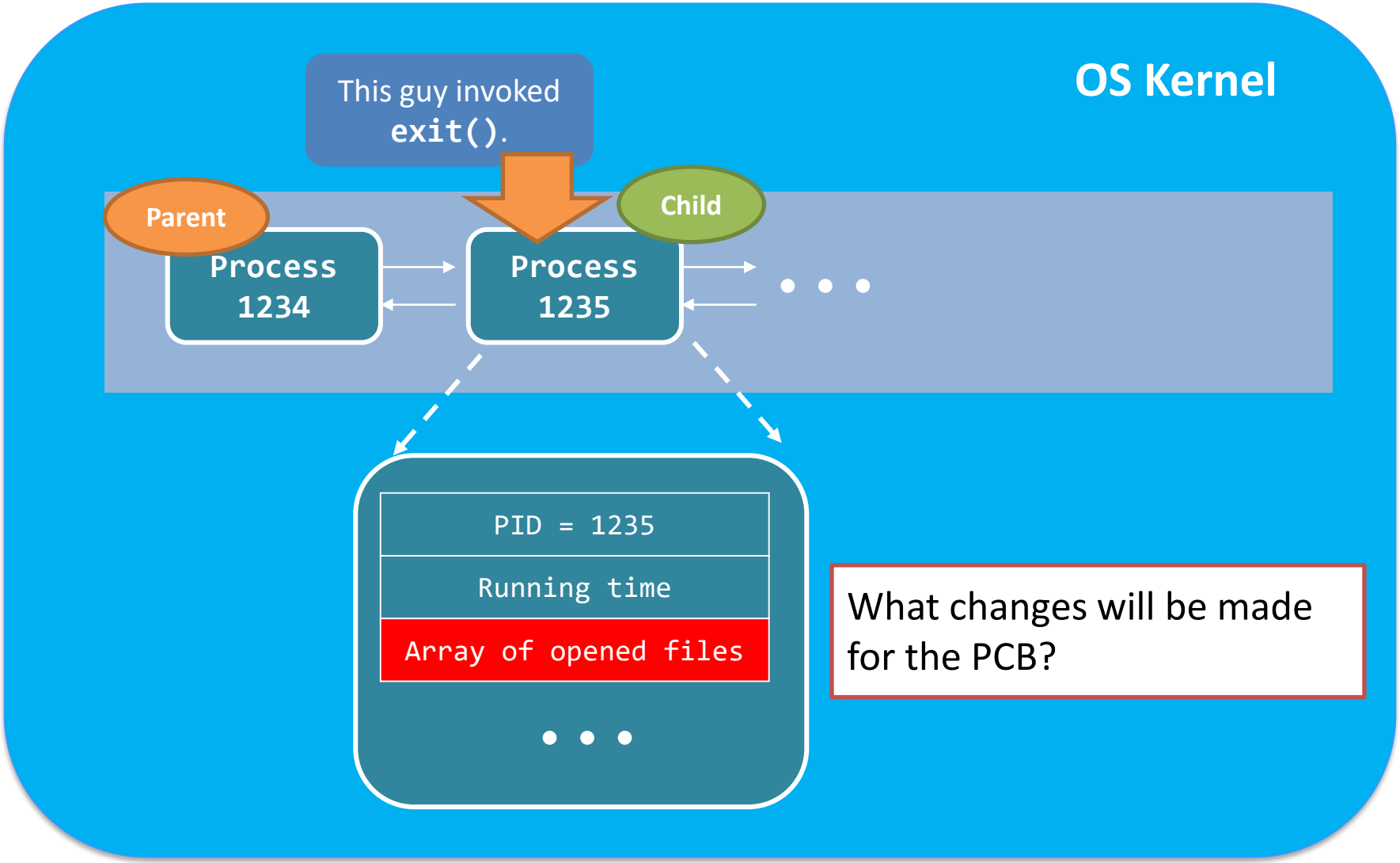
进程终止Wait()和Exit()执行分析



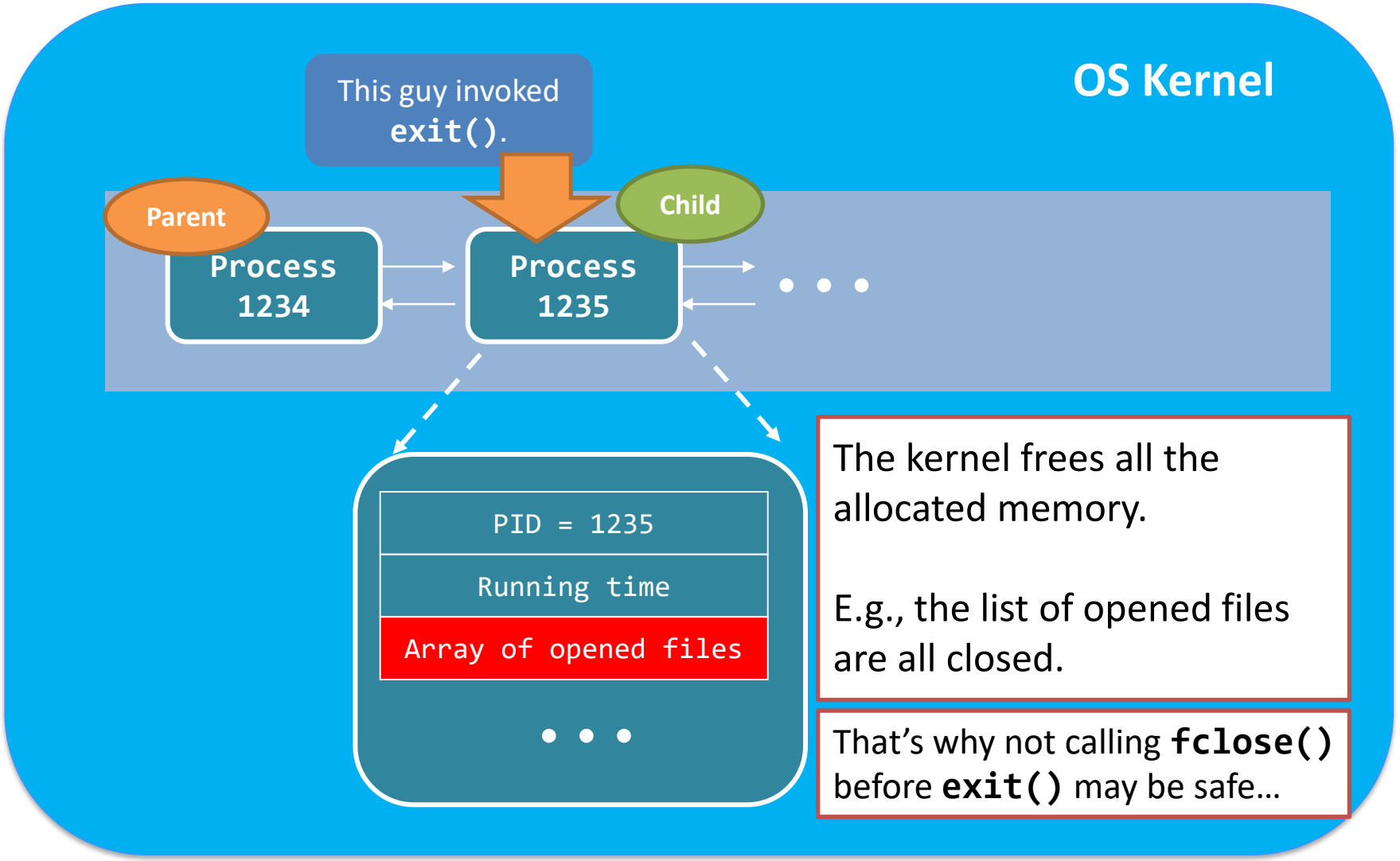
进程终止Wait()和Exit()-从子进程的角度看



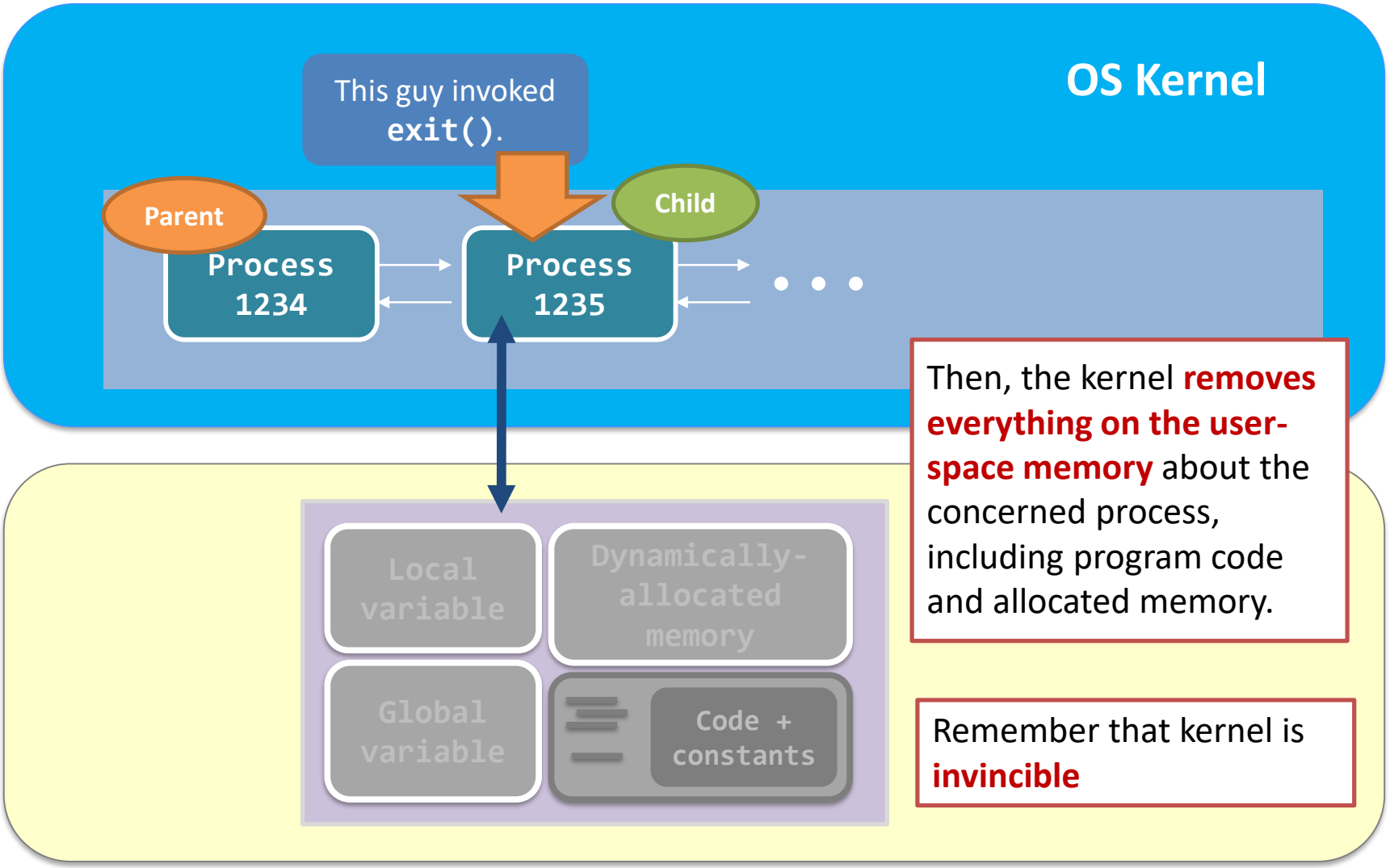
进程终止Wait()和Exit()-从子进程的角度看



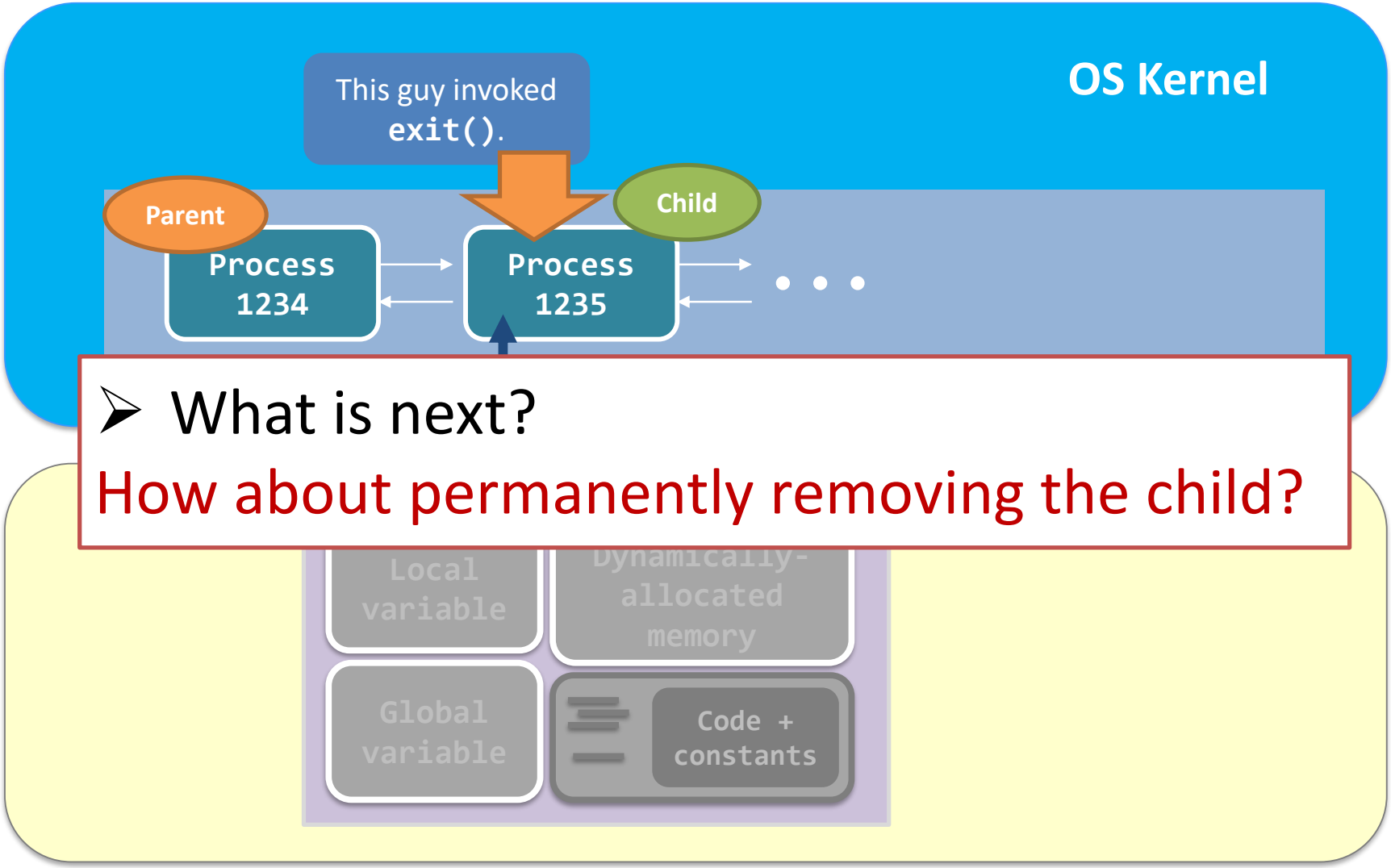
进程终止Wait()和Exit()-从子进程的角度看



进程终止Wait()和Exit()-从子进程的角度看



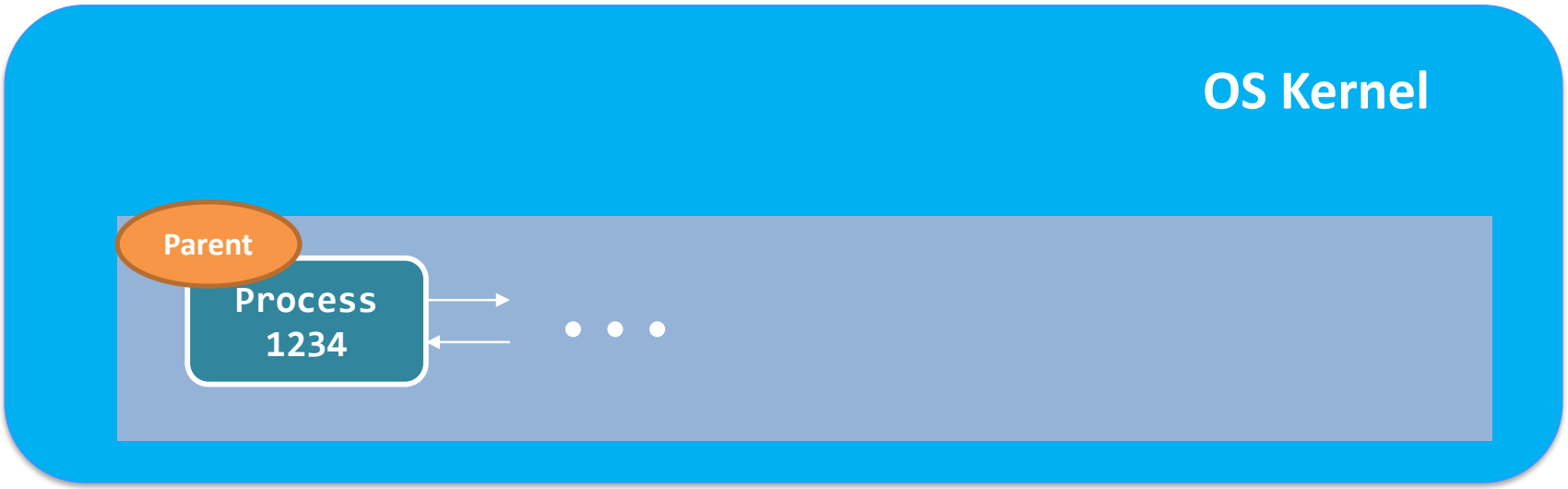
进程终止Wait()和Exit()-从子进程的角度看



- What is next?

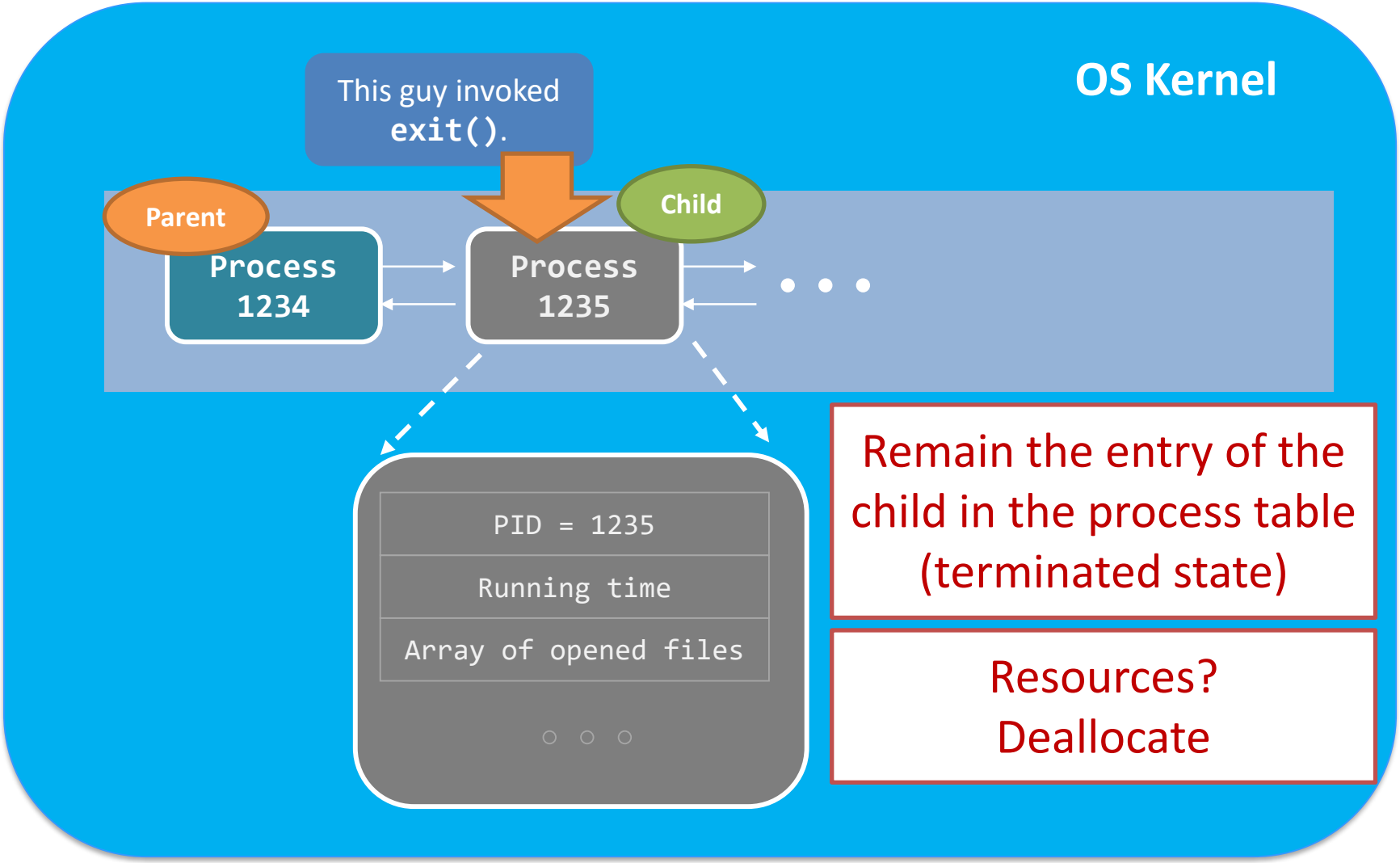
How about permanently removing the child?

进程终止Wait()和Exit()-从子进程的角度看

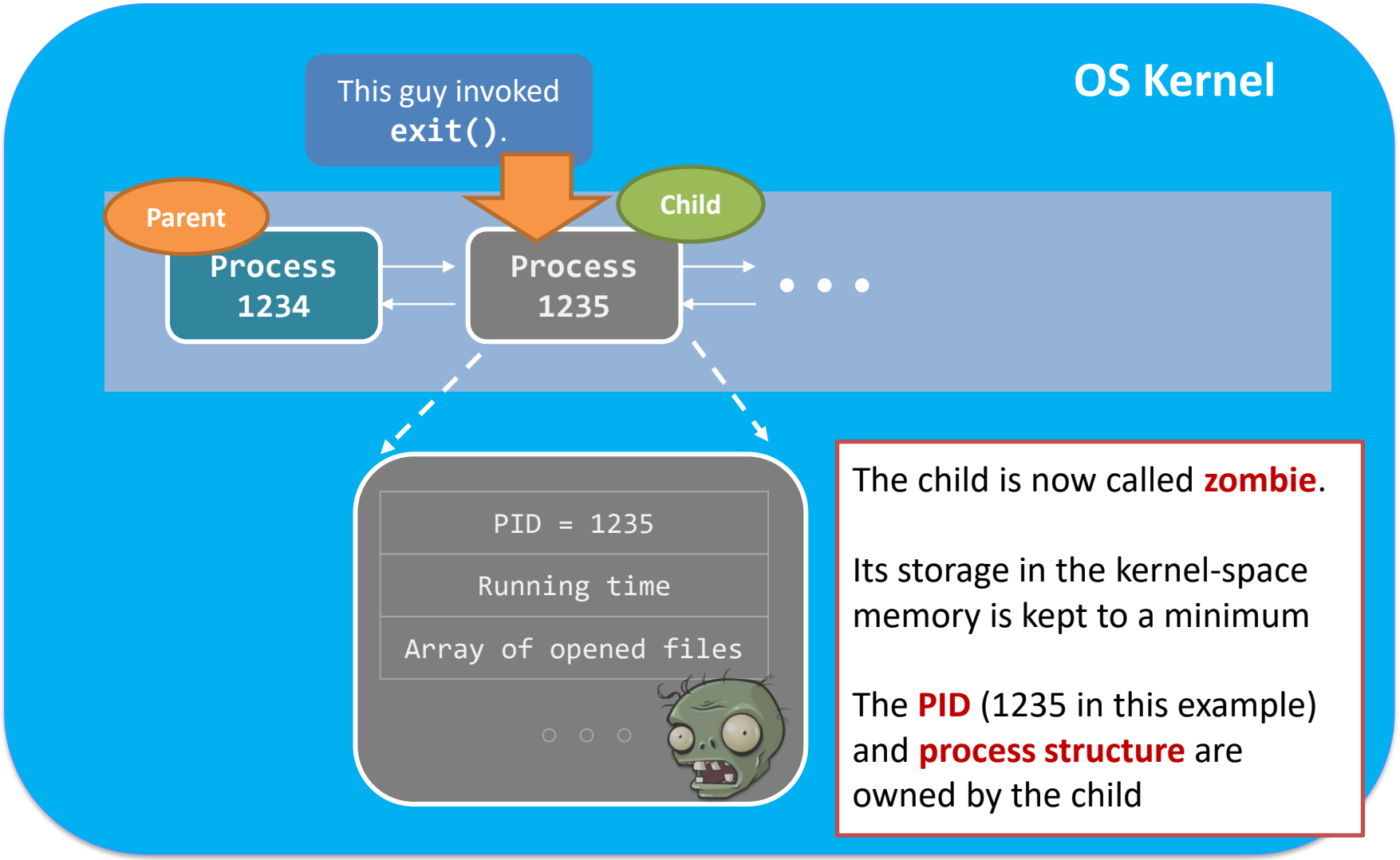


Removed from the process table immediately?
Not really! Why?

进程终止Wait()和Exit()-从子进程的角度看



进程终止Wait()和Exit()-从子进程的角度看

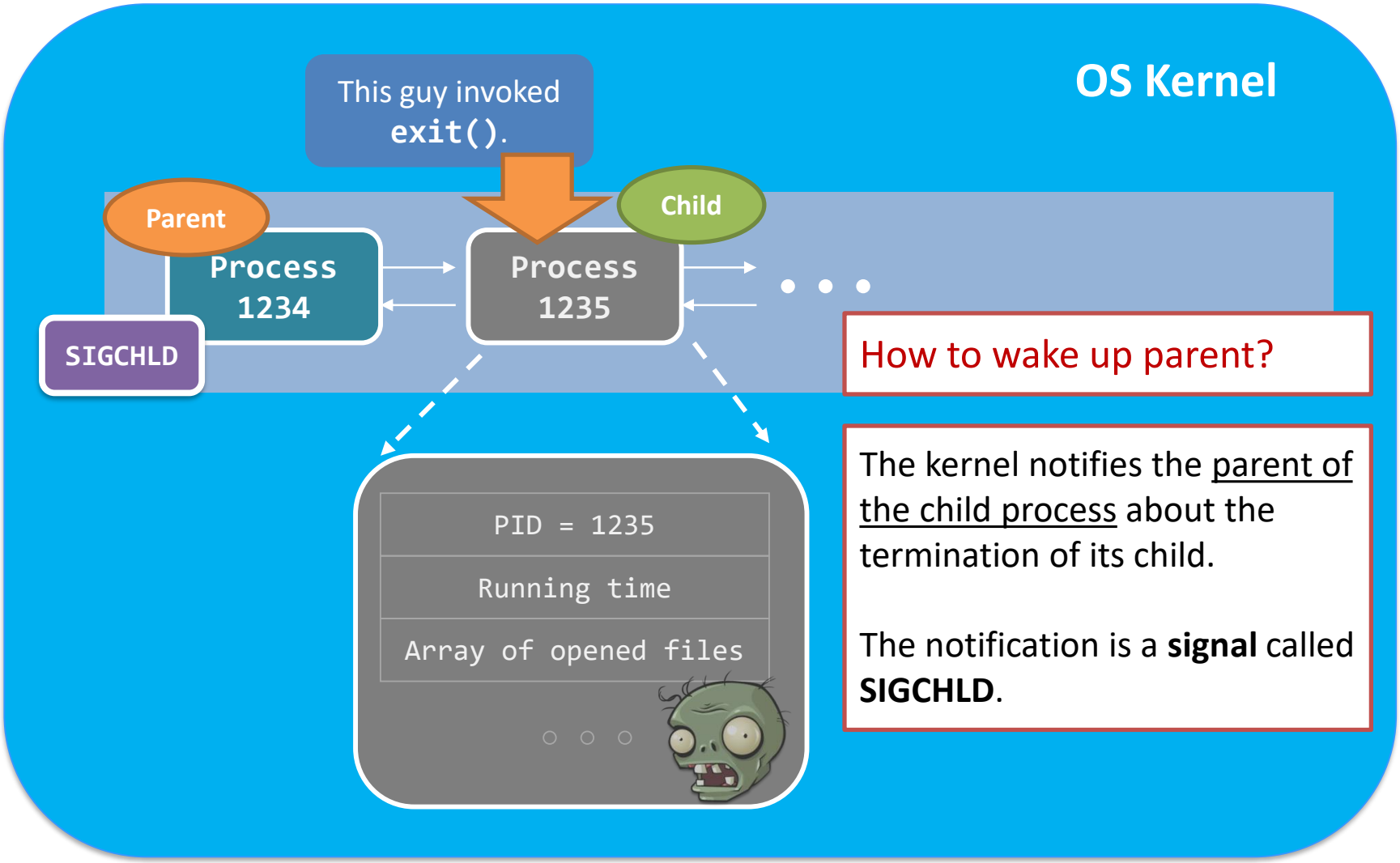


The child is now called **zombie**.

Its storage in the kernel-space memory is kept to a minimum

The **PID** (1235 in this example) and **process structure** are owned by the child

进程终止Wait()和Exit()-从子进程的角度看

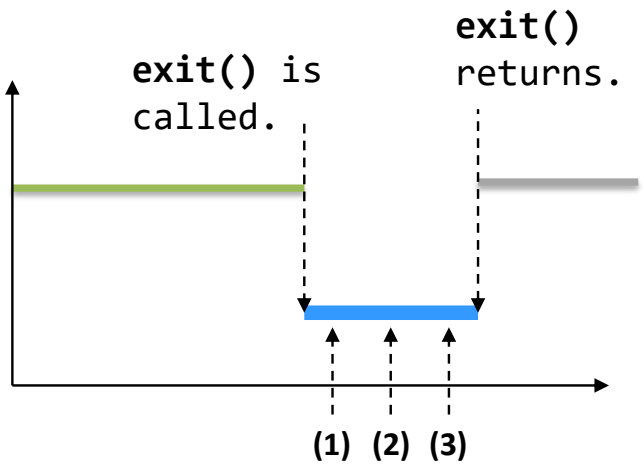


进程终止exit()

Step (1) Clean up most of the allocated kernel-space memory.

Step (2) Clean up all user-space memory.

Step (3) Notify the parent with SIGCHLD.



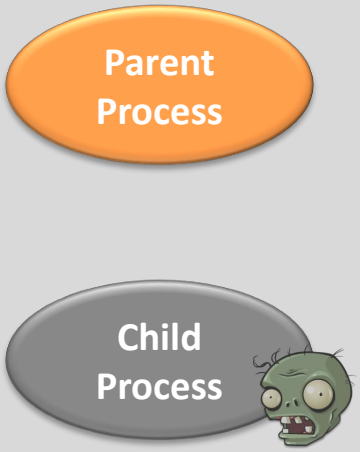
Although the child is still in the system, it is no **longer running**. There is no program code!!!

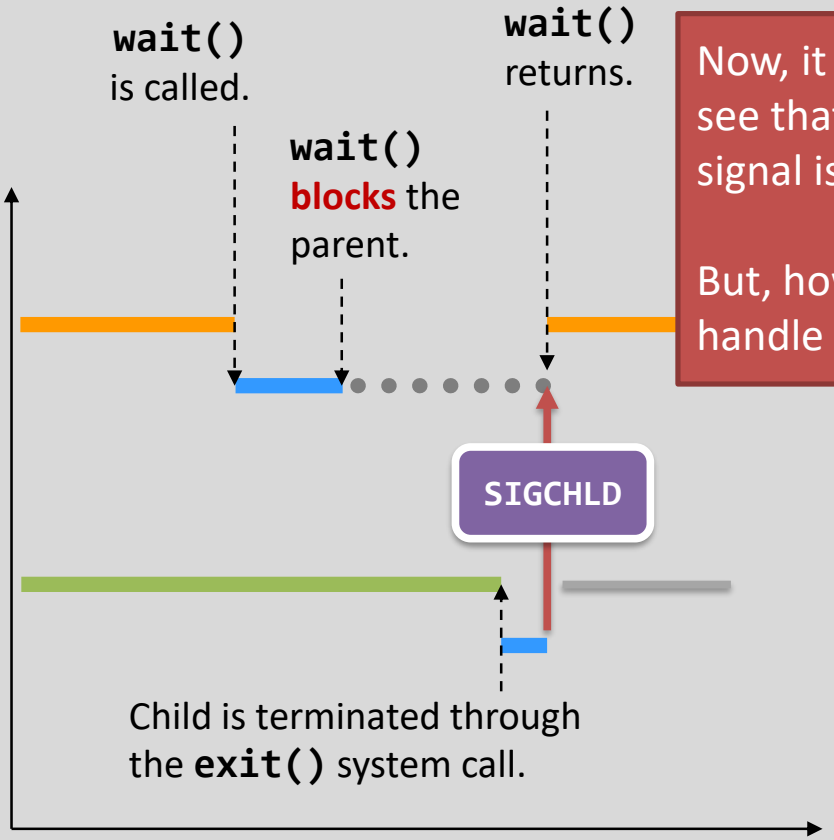
It turns into a **mindless zombie**…

You cannot kill a zombie process, as it is already dead. Then how to eliminate it?

进程终止Wait()+Exit()

How to proceed with wait()?

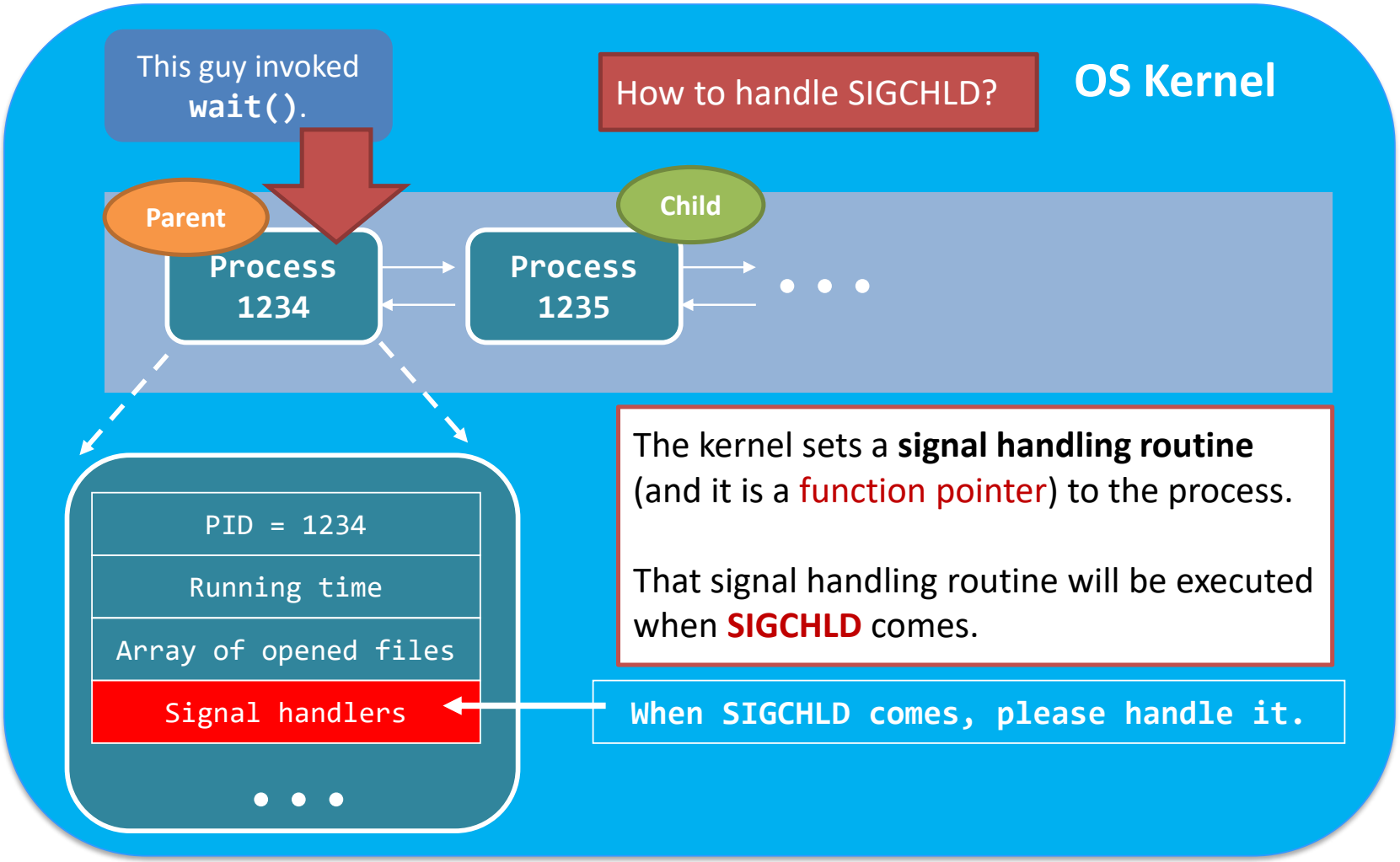




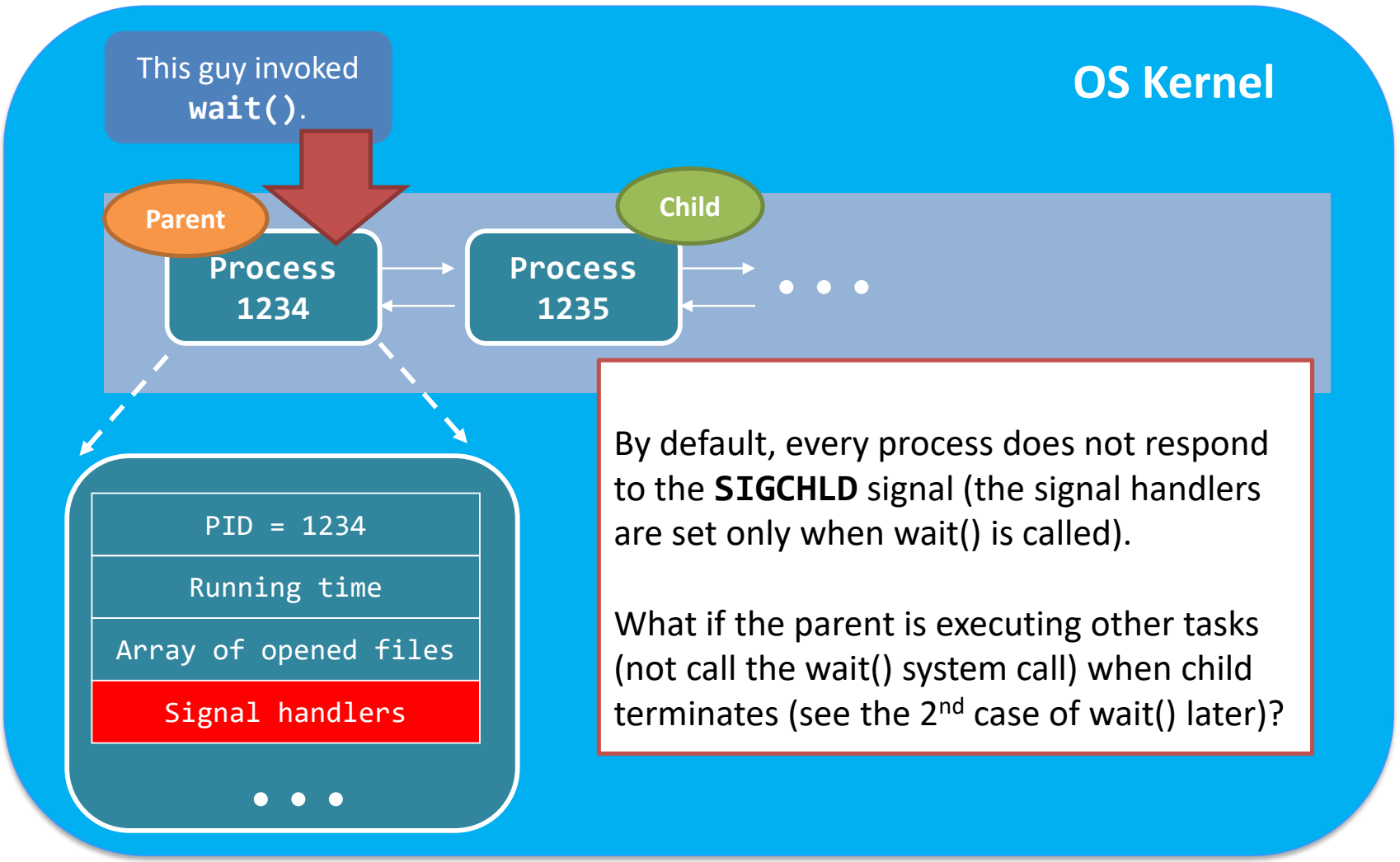
Now, it is trivial to see that **SIGCHLD** signal is the trick!

But, how to handle SIGCHLD?

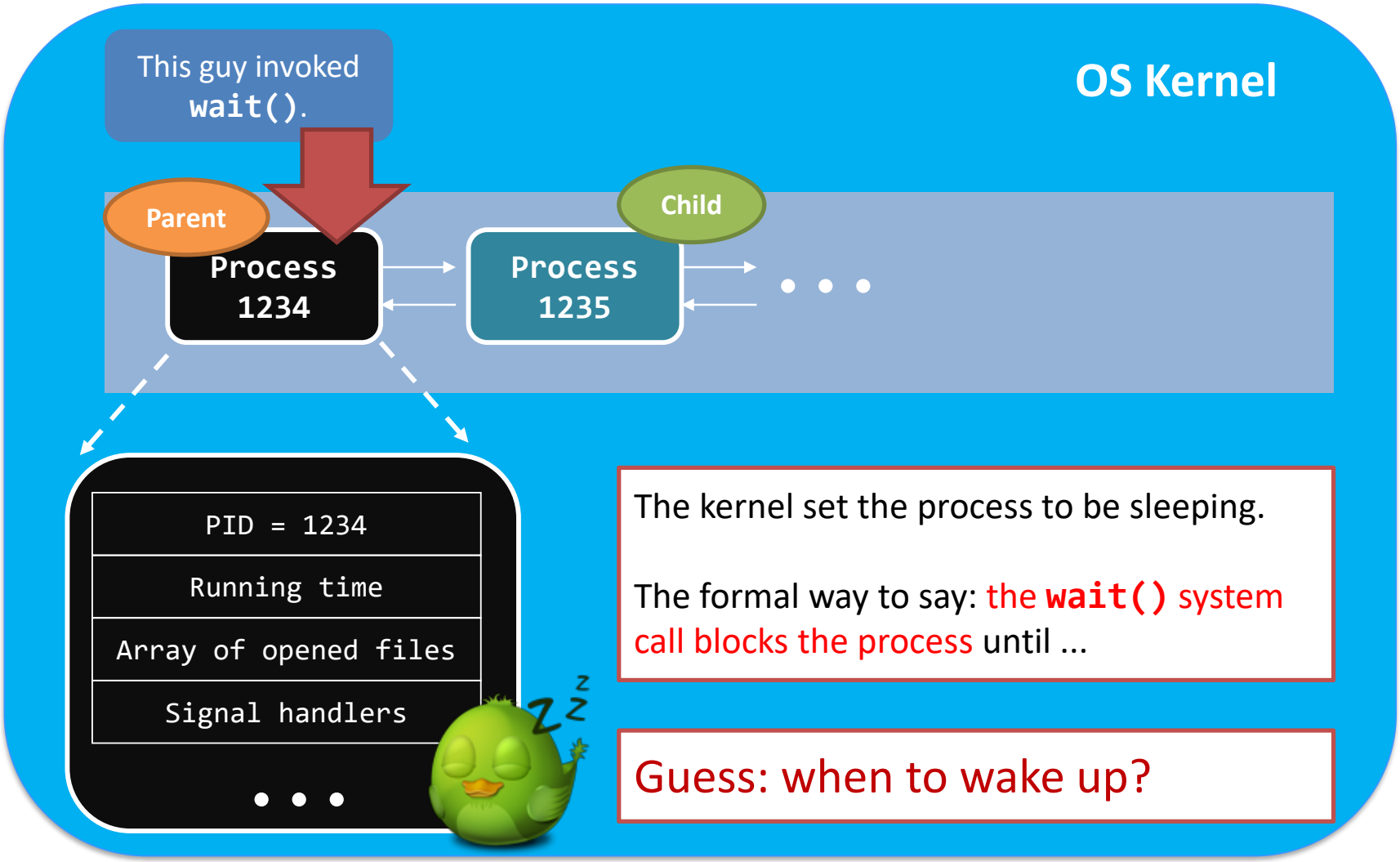
进程终止 Wait()+Exit()-从父进程的角度



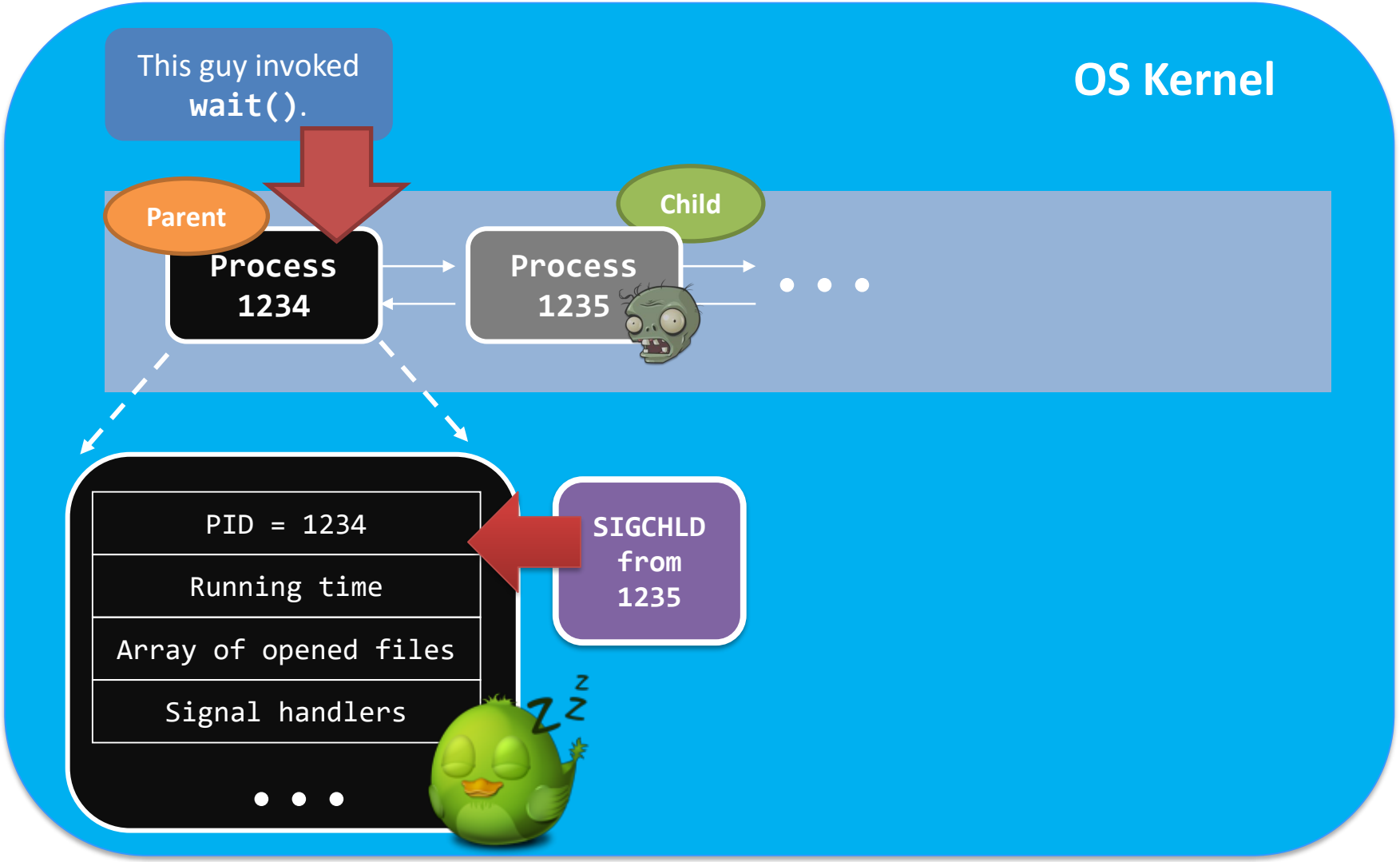
进程终止 Wait()+Exit()-从父进程的角度



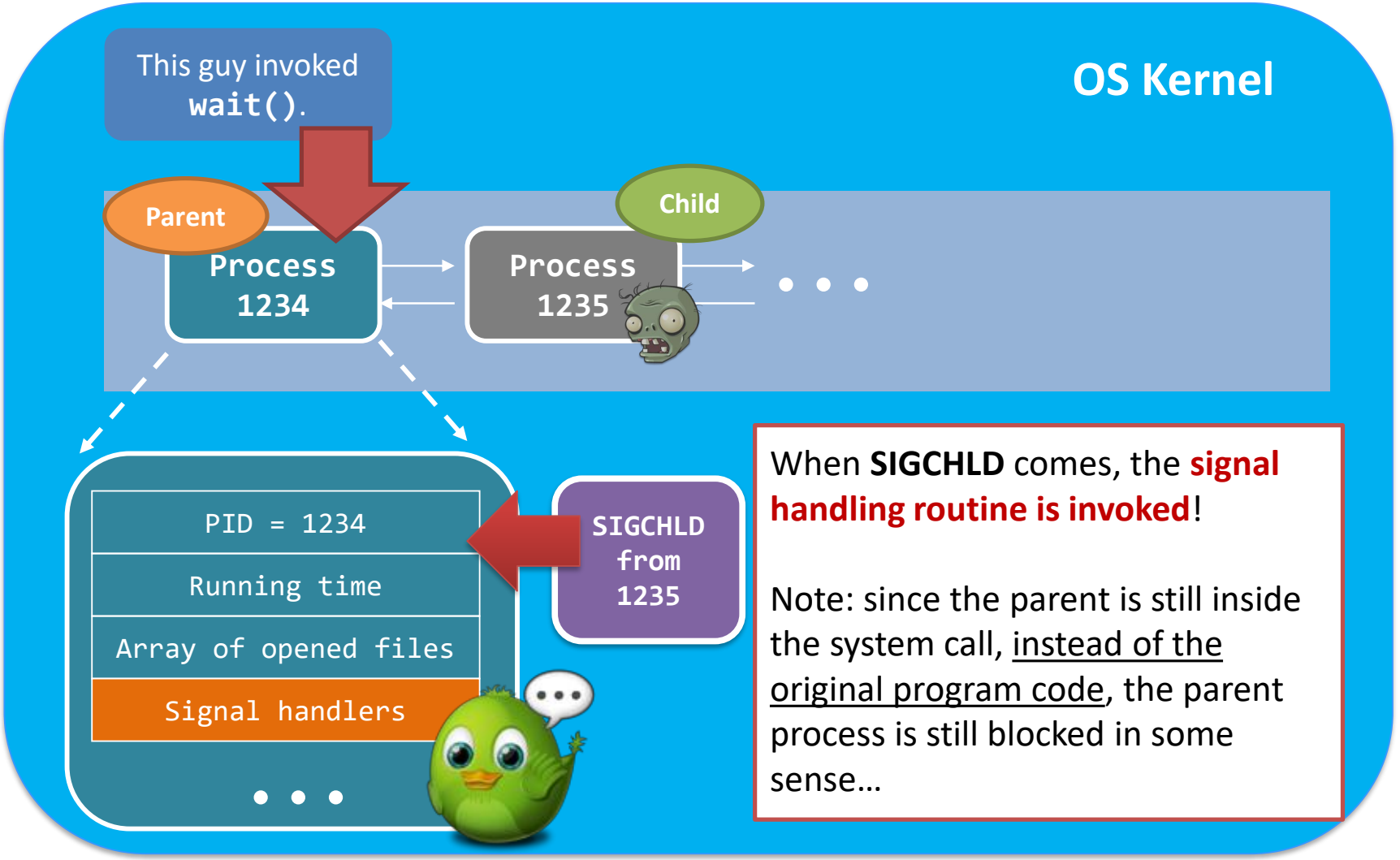
进程终止 Wait()+Exit()-从父进程的角度



进程终止 Wait()+Exit()-从父进程的角度



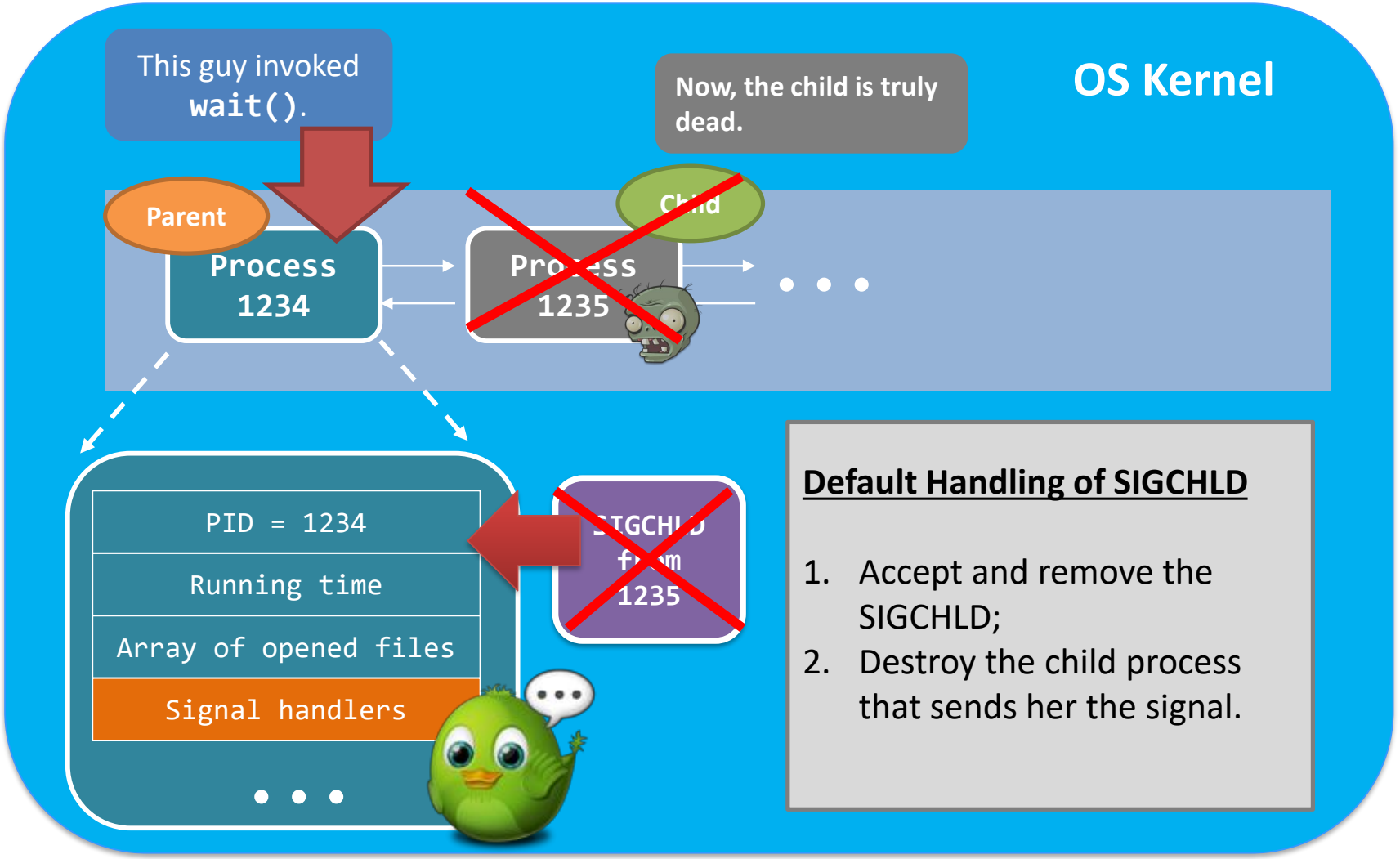
进程终止 Wait()+Exit()-从父进程的角度



When **SIGCHLD** comes, the **signal handling routine is invoked**!

Note: since the parent is still inside the system call, instead of the original program code, the parent process is still blocked in some sense…

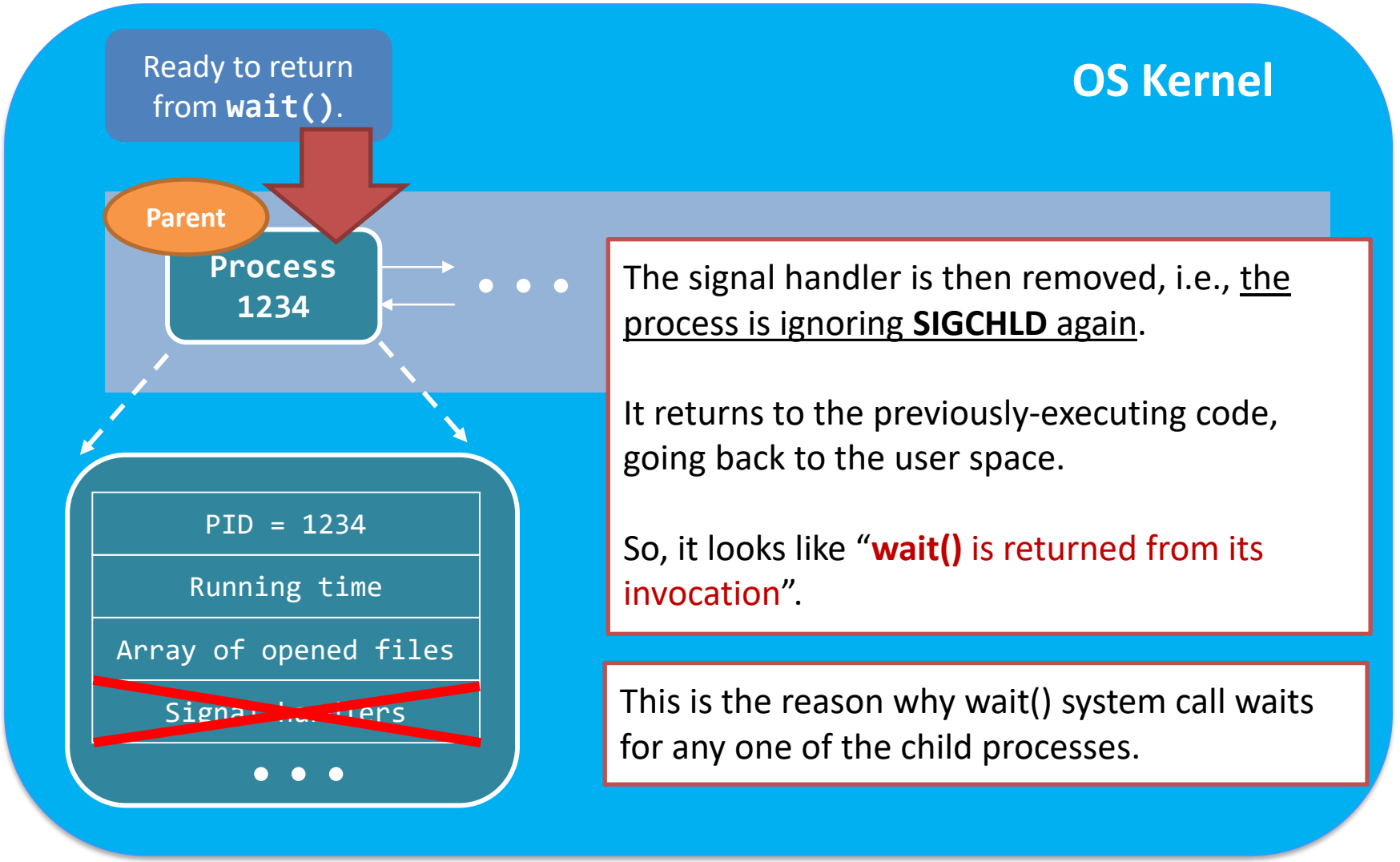
进程终止 Wait()+Exit()-从父进程的角度



Default Handling of SIGCHLD

1. Accept and remove the SIGCHLD;
2. Destroy the child process that sends her the signal.

进程终止 Wait()+Exit()-从父进程的角度



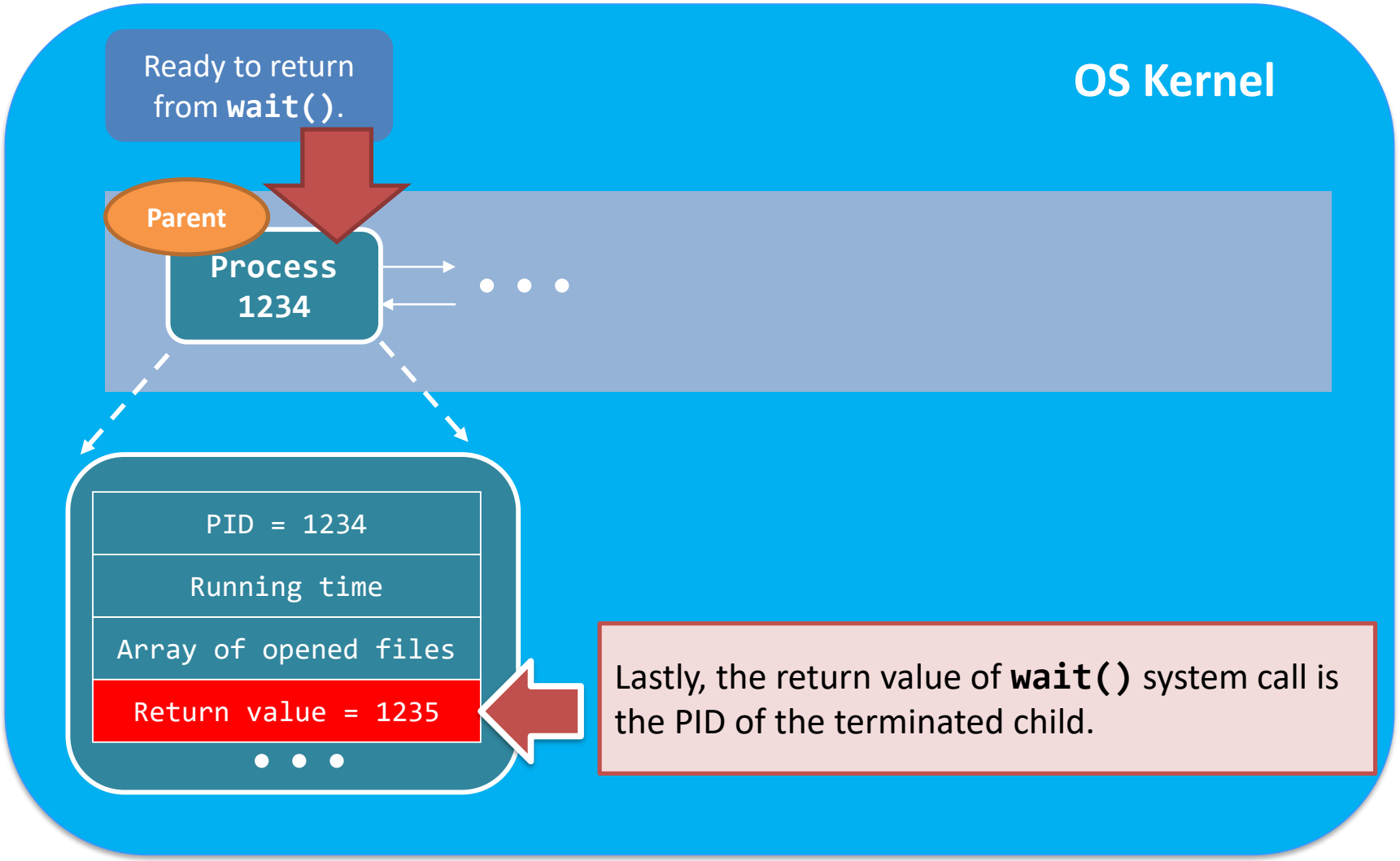
The signal handler is then removed, i.e., the process is ignoring **SIGCHLD** again.

It returns to the previously-executing code, going back to the user space.

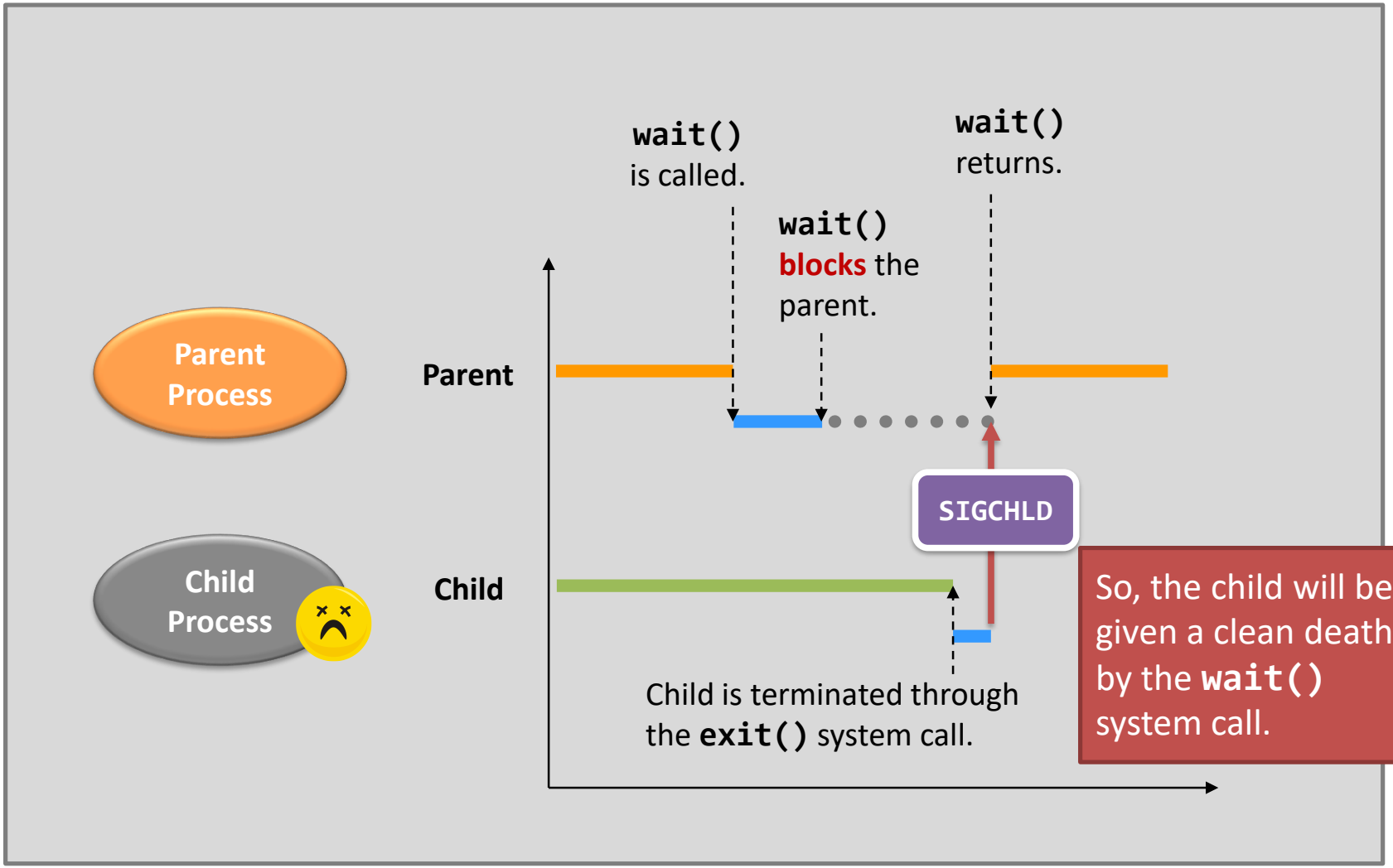
So, it looks like "**wait()** is returned from its invocation".

This is the reason why wait() system call waits for any one of the child processes.

进程终止 Wait()+Exit()-从父进程的角度



进程终止 Wait()+Exit()-从父进程的角度



 如果父进程在子进程终止后调用wait()结果如何?

多进程体系结构 - Chrome浏览器

- 许多web浏览器作为单个进程运行（有些仍然如此）
  - 如果一个网站出现问题，整个浏览器可能会挂起或崩溃

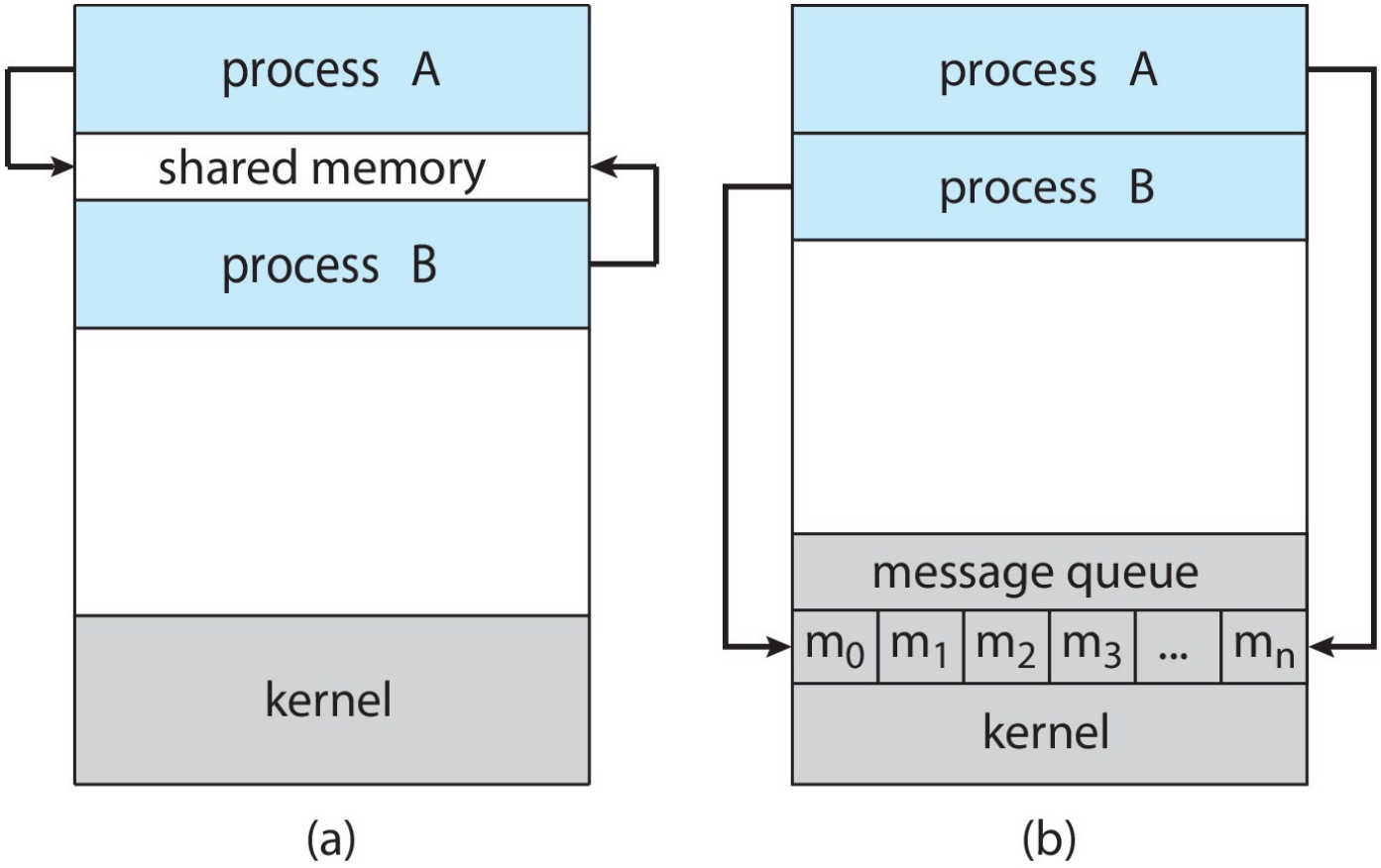
- Google Chrome浏览器是多进程的，有3种不同类型的进程：
  - 浏览器进程管理用户界面、磁盘和网络I/O
  - 渲染器进程渲染网页，处理HTML、Javascript。为每个打开的网站创建一个新的渲染器
    - 在沙箱中运行，限制磁盘和网络I/O，最大限度地减少安全漏洞攻击的影响
  - 每种类型插件的插件进程

进程间通信

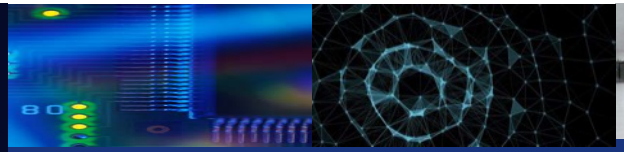
- 系统内的过程可以是独立的，也可以是协作的
- 协作进程可以影响或受其他进程的影响，包括共享数据
- 合作进程的原因：
  - 信息共享
  - 计算加速
  - 模块化
  - 方便用户
- 协作进程需要进程间通信（IPC）
- IPC的两种模型
  - 共享内存
  - 消息传递

进程间通信模型

（a） 共享内存。（b）信息传递。

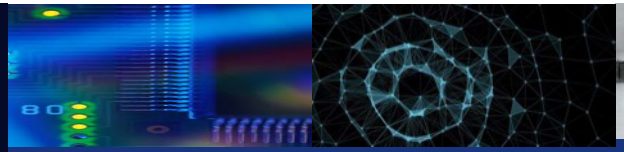


多核处理器系统中，消息传递优于共享内存，因为没有高速缓存（Cache）的一致性问题。



共享内存系统

- 希望通信的进程之间共享的内存区域，通信双方需要同时映射共享内存到自己的进程内；
- 通信由用户进程控制，而不是由操作系统控制；
- 主要问题是提供一种机制，允许用户进程在访问共享内存时同步其操作；
- 第6章和第7章详细讨论了同步；



生产者消费者问题

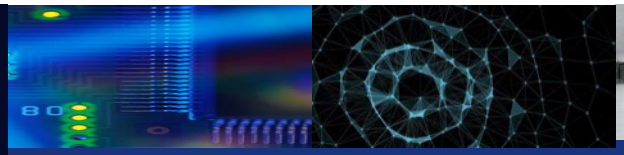
- **协作进程的范例：**
  - 生产者进程产生消费者进程所消费的信息；
  - 例如Web服务器和客户端；
  - 消息中间件；
- **两种变体：**
  - 无边界缓冲区对缓冲区的大小没有实际限制：
    - 生产者从不等待；
    - 如果没有要使用的缓冲区，则消费者将等待；
  - 有界缓冲区假定存在固定的缓冲区大小：
    - 如果所有缓冲区都已满，则生产者必须等待；
    - 如果没有可用的缓冲区，则消费者将等待；

有界缓冲区-共享内存解决方案

- 共享数据结构

```
#define BUFFER_SIZE 10

typedef struct {
    … …    /* item structure */
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

共享buffer的实现采用一个循环数组和两个逻辑指针：in和out。变量in 指向缓冲区的下一个空位；变量out指向缓冲区的第一个满位。当in==out时，缓冲区为空；当(in+1)%BUFFER_SIZE==out时，缓冲区满。



有界缓冲区-共享内存解决方案

- 生产者

```
item next_produced;
while (true) {
        … …     /* produce an item saved in next_produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ;     /* buffer full, do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

- 消费者

```
item next_consumed;
while (true) {
    while (in == out)
        ;      /* buffer empty, do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
            /* consume the item in next_consumed */
}
```

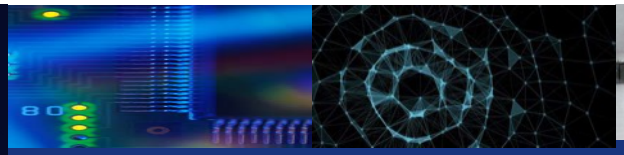
如何处理多个生产者、消费者访问共享内存的问题?

Linux共享内存

- Linux IPC 限制

  限制信息在/etc/sysctl.conf文件中

```
------ Messages Limits --------
max queues system wide = 32000
max size of message (bytes) = 8192
default max size of queue (bytes) = 16384

------ Shared Memory Limits --------
max number of segments = 4096
max seg size (kbytes) = 18014398509465599
max total shared memory (kbytes) = 18014398509481980
min seg size (bytes) = 1

------ Semaphore Limits --------
max number of arrays = 32000
max semaphores per array = 32000
max semaphores system wide = 1024000000
max ops per semop call = 500
semaphore max value = 32767
```



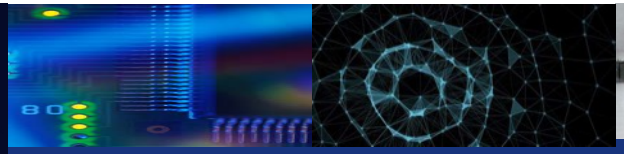

Linux共享内存

- **Key ID**

```
#include <sys/shm.h>
key_t ftok(const char *pathname, int id);
/* key_t is of type int. ftok() convert a pathname and a project identifie
to an IPC key */
key_t key = ftok(“/home/myshm", 0x27);
if((key == -1) {
    perror(“ftok()");
} else
    printf("key = 0x%x\n", key);
```

- **Create**

```
int shmget(key_t key, int size, int shmflg);
/* shmget() allocates a shared memory segment */
/* upper bound of size: 1.9G */

int shmid = shmget(IPC_PRIVATE, 4096, IPC_CREATE|IPC_EXCL|0660);
if(shmid == -1) {
    perror("shmget()");
}
```



Linux共享内存

- **Attach**

```
void *shmat(int shmid, const void *shmaddr, int shmflg);

void *shmptr = shmat(shmid, 0, 0);
    /* shmaddr=0: attaching address is decided by kernel */
if(shmptr == (void *)(-1))
    perror("shmat()");
```

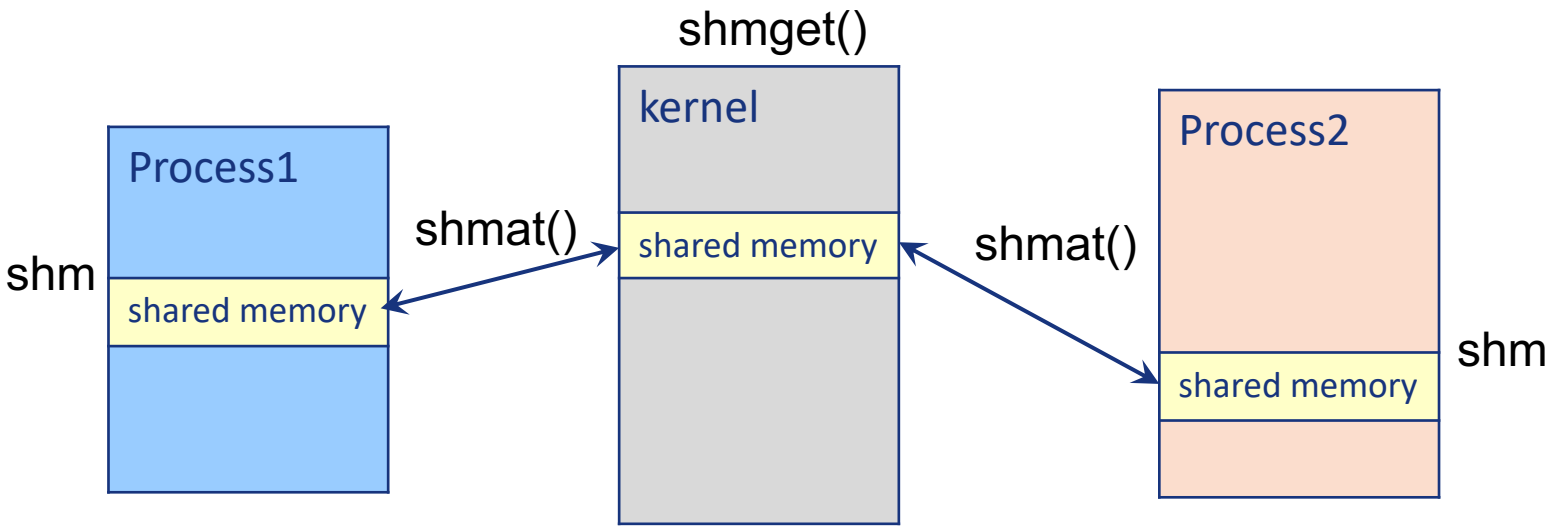
- **Detach**

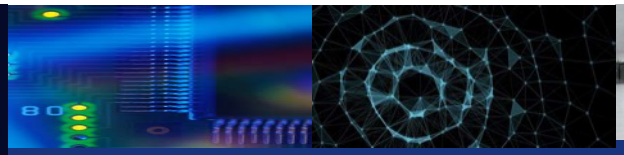
```
int shmdt(const void *shmaddr);

if(shmdt(shmptr) == -1)
    perror("shmdt()");
```

Linux共享内存

- **Release**

```
int shmctl(int shmid,int cmd,struct shmid_ds *buf);

if (shmctl(shmid, IPC_RMID, 0) == -1)
    perror("shmctl()");
```

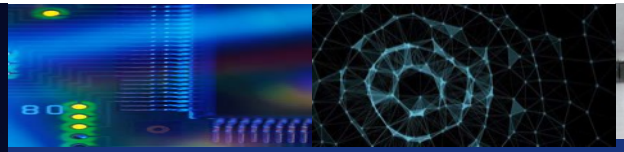





Linux共享内存

- Single-writer-single-reader problem illustrating shared-memory, define shmdata.h

```
#define TEXT_SIZE 4*1024  /* = PAGE_SIZE, size of each message */
#define TEXT_NUM 1      /* maximal number of messages */
    /* total size can not exceed current shmmax,
       or an 'invalid argument' error occurs when shmget */
#define PERM S_IRUSR|S_IWUSR|IPC_CREAT

#define ERR_EXIT(m) \
    do { \
        perror(m); \
        exit(EXIT_FAILURE); \
    } while(0)

/* a demo structure, modified as needed */
struct shared_struct {
    int written; /* flag = 0: buffer writable; others: readable */
    char mtext[TEXT_SIZE]; /* buffer for message reading and writing */
};
```

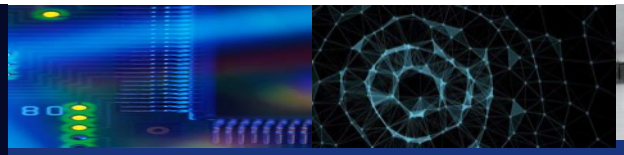


Linux共享内存

shmcon.c

```
int main(int argc, char *argv[])
{
    struct stat fileattr;
    key_t key; /* of type int */
    int shmid; /* shared memory ID */
    void *shmptr;
    struct shared_struct *shared; /* structured shm */
    pid_t childpid1, childpid2;
    char pathname[80], key_str[10], cmd_str[80];
    int shmsize, ret;

    shmsize = TEXT_NUM*sizeof(struct shared_struct);
    printf("max record number = %d, shm size = %d\n", TEXT_NUM, shmsize);

    if(argc <2) {
        printf("Usage: ./a.out pathname\n");
        return EXIT_FAILURE;
    }
    strcpy(pathname, argv[1]);
    if(stat(pathname, &fileattr) == -1) {
        ret = creat(pathname, O_RDWR);
        if (ret == -1) {
            ERR_EXIT("creat()");
        }
        printf("shared file object created\n");
    }
```

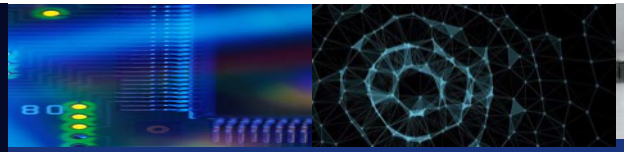
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <sys/shm.h>
#include <fcntl.h>
#include “alg.8-0-shmdata.h"
```



Linux共享内存

shmcon.c

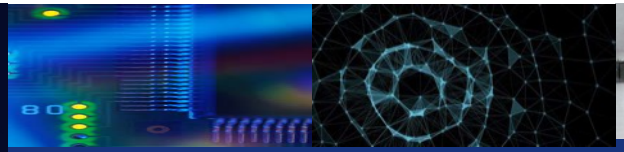
```
key = ftok(pathname, 0x27); /* 0x27 a pro_id 0x0001 - 0xffff, 8 least bits used */
    if(key == -1) {
        ERR_EXIT("shmcon: ftok()");
    }
    printf("key generated: IPC key = 0x%x\n", key); /* set any key>0 without ftok() */

    shmid = shmget((key_t)key, shmsize, 0666|PERM);
    if(shmid == -1) {
        ERR_EXIT("shmcon: shmget()");
    }
    printf("shmcon: shmid = %d\n", shmid);

    shmptr = shmat(shmid, 0, 0); /* returns the virtual base address mapping to the
shared memory, *shmaddr=0 decided by kernel */

    if(shmptr == (void *)-1) {
        ERR_EXIT("shmcon: shmat()");
    }
    printf("shmcon: shared Memory attached at %p\n", shmptr);

    shared = (struct shared_struct *)shmptr;
    shared->written = 0;

    sprintf(cmd_str, "ipcs -m | grep '%d'\n", shmid);
    printf("\n------ Shared Memory Segments ------\n");
    system(cmd_str);
```



Linux共享内存

shmcon.c

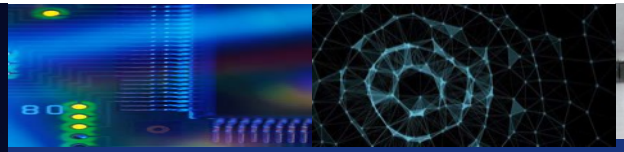
```
if(shmdt(shmptr) == -1) {
        ERR_EXIT("shmcon: shmdt()");
    }

    printf("\n------ Shared Memory Segments ------\n");
    system(cmd_str);

    sprintf(key_str, "%x", key);
    char *argv1[] = {" ", key_str, 0};

    childpid1 = vfork();
    if(childpid1 < 0) {
        ERR_EXIT("shmcon: 1st vfork()");
    }
    else if(childpid1 == 0) {
        execv("./alg.8-2-shmread.o", argv1); /* call shm_read with IPC key */
    }
    else {
        childpid2 = vfork();
        if(childpid2 < 0) {
            ERR_EXIT("shmcon: 2nd vfork()");
        }
        else if (childpid2 == 0) {
            execv("./alg.8-3-shmwrite.o", argv1); /* call shmwrite with IPC key
*/
        }
```



Linux共享内存

shmcon.c

```
else {
            wait(&childpid1);
            wait(&childpid2);
                 /* shmid can be removed by any process known the
IPC key */
            if (shmctl(shmid, IPC_RMID, 0) == -1) {
                ERR_EXIT("shmcon: shmctl(IPC_RMID)");
            }
            else {
                printf("shmcon: shmid = %d removed \n", shmid);
                printf("\n------ Shared Memory Segments ------
\n");
                system(cmd_str);
                printf("nothing found ...\n");
                return EXIT_SUCCESS;
            }
        }
    }
}
```

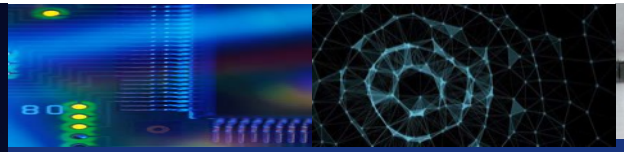


Linux共享内存

shmread.c (1)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>
#include <sys/shm.h>
#include “alg.8-0-shmdata.h"
```

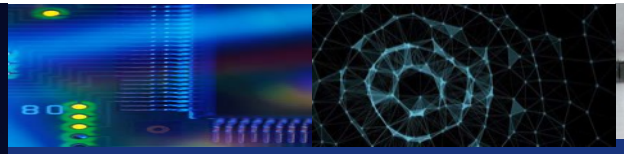
```
int main(int argc, char *argv[])
{
    void *shmptr = NULL;
    struct shared_struct *shared;
    int shmid;
    key_t key;

    sscanf(argv[1], “%x", &key);
    printf("%*sshmread: IPC key = 0x%x\n", 30, " ", key);

    shmid = shmget((key_t)key, TEXT_NUM*sizeof(struct shared_struct),
0666|PERM);
    if (shmid == -1) {
        ERR_EXIT("shread: shmget()");
    }

    shmptr = shmat(shmid, 0, 0);
    if(shmptr == (void *)-1) {
        ERR_EXIT("shread: shmat()");
    }
    printf("%*sshmread: shmid = %d\n", 30, " ", shmid);
    printf("%*sshmread: shared memory attached at %p\n", 30, " ", shmptr);
    printf("%*sshmread process ready ...\n", 30, " ");

    shared = (struct shared_struct *)shmptr;
```



Linux共享内存

shmread.c (1)

```
 while (1) {
        while (shared->written == 0) {
            sleep(1); /* message not ready, waiting ... */
        }
        printf("%*sYou wrote: %s\n", 30, " ", shared->mtext);
        shared->written = 0;
        if (strncmp(shared->mtext, "end", 3) == 0) {
            break;
        }
    } /* it is not reliable to use shared->written for process
synchronization */

   if (shmdt(shmptr) == -1) {
        ERR_EXIT("shmread: shmdt()");
   }

    sleep(1);
    exit(EXIT_SUCCESS);
}
```

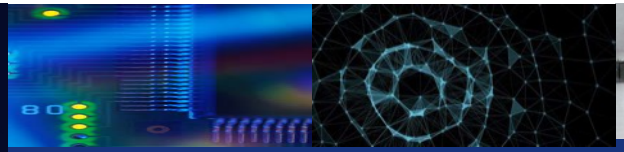


Linux共享内存

shmwrite.c (1)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>
#include <sys/shm.h>
#include “alg.8-0-shmdata.h"
```

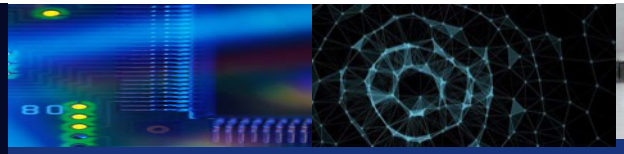
```
int main(int argc, char *argv[])
{
    void *shmptr = NULL;
    struct shared_struct *shared = NULL;
    int shmid;
    key_t key;

    char buffer[BUFSIZ + 1]; /* 8192bytes, saved from stdin */

    sscanf(argv[1], "%x", &key);
    printf("shmwrite: IPC key = 0x%x\n", key);

    shmid = shmget((key_t)key, TEXT_NUM*sizeof(struct shared_struct), 0666|PERM);
    if (shmid == -1) {
        ERR_EXIT("shmwite: shmget()");
    }

    shmptr = shmat(shmid, 0, 0);
    if(shmptr == (void *)-1) {
        ERR_EXIT("shmwrite: shmat()");
    }
    printf("shmwrite: shmid = %d\n", shmid);
    printf("shmwrite: shared memory attached at %p\n", shmptr);
    printf("shmwrite precess ready ...\n");

    shared = (struct shared_struct *)shmptr;
```



Linux共享内存

shmwrite.c (1)

```
while (1) {
        while (shared->written == 1) {
            sleep(1); /* message not read yet, waiting ... */
        }

        printf("Enter some text: ");
        fgets(buffer, BUFSIZ, stdin);
        strncpy(shared->mtext, buffer, TEXT_SIZE);
        printf("shared buffer: %s\n",shared->mtext);
        shared->written = 1;  /* message prepared */

        if(strncmp(buffer, "end", 3) == 0) {
            break;
        }
    }
       /* detach the shared memory */
    if(shmdt(shmptr) == -1) {
        ERR_EXIT("shmwrite: shmdt()");
    }

    sleep(1);
    exit(EXIT_SUCCESS);
}
```



POSIX Shared Memory

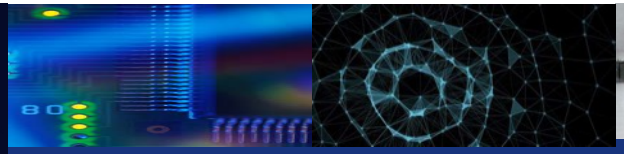
- Several IPC mechanisms are available for POSIX systems, including shared memory and message passing.
- POSIX shared memory is organized using *memory-mapped files*, which associate the region of shared memory with a file in /dev/shm/.
- For memory sharing, a process must first create a shared-memory object using the shm_open() system call:

```
int shm_open(const char *path, int flags, mode_t mode);
```

  - Example.

```
fd = shm_open(name, O_CREAT|O_RDWR, 0666);
```

  - *path*: the name of the shared-memory object. Processes that wish to access this shared memory must refer to the object by this name.
  - *flags*: the shared-memory object is to be created if it does not yet exist (O_CREAT) and that the object is open for reading and writing (O_RDWR).
  - *mode*: the file-access permissions of the shared-memory object.
  - A successful call to shm_open() returns an integer file descriptor for the shared-memory object.



POSIX Shared Memory

- Once the object is established, the ftruncate() function is used to configure the size of the object in bytes.

```
int ftruncate(int fd, off_t length);
```

  - E.g., the following call sets the size of the object to 4,096 bytes.

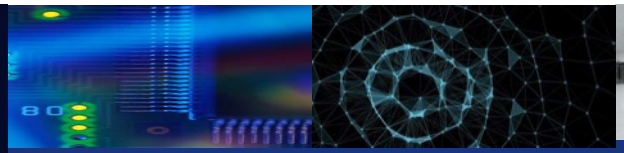
```
ftruncate(fd, 4096);
```

- Finally, the mmap() function establishes a memory-mapped file containing the shared-memory object. It also returns a pointer to the memory-mapped file that is used for accessing the shared-memory object.

```
void *mmap(void *addr, size_t len, int prot, int flags,
int fd, off_t offset);
```

- The API is supported by Linux 2.4 and later, FreeBSD, …
- Compiling

```
gcc -lrt filename.c
```

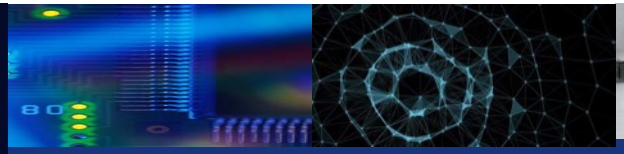


- Producer-Consumer problem illustrating POSIX shared-memory API.

shmpthreadcon.c (1)

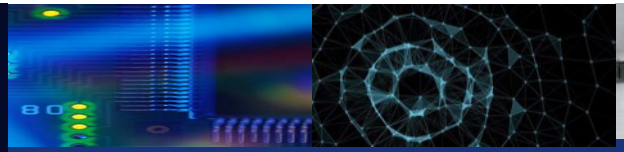
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <sys/mman.h>

#include "shmdata.h"
```

```
/* gcc -lrt */
int main(int argc, char *argv[])
{
    char pathname[80], cmd_str[80];
    struct stat fileattr;
    int fd, shmsize, ret;
    pid_t childpid1, childpid2;
    if(argc < 2) {
        printf("Usage: ./a.out filename\n");
        return EXIT_FAILURE;
    }
    fd = shm_open(argv[1], O_CREAT|O_RDWR, 0666);
        /* /dev/shm/filename as the shared object, creating if not exist */
    if(fd == -1) {
        ERR_EXIT("con: shm_open()");
    }
    system("ls -l /dev/shm/");
    shmsize = TEXT_NUM*sizeof(struct shared_struct);
    ret = ftruncate(fd, shmsize);
    if(ret == -1) {
        ERR_EXIT("con: ftruncate()");
```



shmpthreadcon.c (2)

```
char *argv1[] = {" ", argv[1], 0};
    childpid1 = vfork();
    if(childpid1 < 0) {
        ERR_EXIT("shmpthreadcon: 1st vfork()");
    }
    else if(childpid1 == 0) {
        execv("./shmproducer", argv1); /* call producer with filename */
    }
    else {
        childpid2 = vfork();
        if(childpid2 < 0)
            ERR_EXIT("shmpthreadcon: 2nd vfork()");
        else if (childpid2 == 0)
            execv("shmconsumer.o", argv1); /* call consumer with filename */
        else {
            wait(&childpid1);
            wait(&childpid2);
            ret = shm_unlink(argv[1]);
            if(ret == -1) {
                ERR_EXIT(“con: shm_unlink()");
            } /* shared object can be removed by any process knew the filename */
            system("ls -l /dev/shm/");
        }
    }
    exit(EXIT_SUCCESS);
}
```

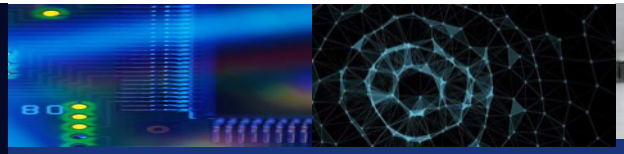


Producer-Consumer problem illustrating POSIX shared-memory API.

shmproducer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>

#include "shmdata.h"
```

```
/* gcc -lrt */
int main(int argc, char *argv[])
{
    int fd, shmsize, ret;
    void *shmptr;
    const char *message_0 = "Hello World!";

    fd = shm_open(argv[1], O_RDWR, 0666); /* /dev/shm/filename as the
shared object */
    if(fd == -1) {
        ERR_EXIT("producer: shm_open()");
    }

    shmsize = TEXT_NUM*sizeof(struct shared_struct);
    shmptr = (char *)mmap(0, shmsize, PROT_READ|PROT_WRITE, MAP_SHARED, fd,
0);
    if(shmptr == (void *)-1) {
        ERR_EXIT("producer: mmap()");
    }

    sprintf(shmptr,"%s",message_0);
    printf("produced message: %s\n", (char *)shmptr);

    return EXIT_SUCCESS;
}
```



Producer-Consumer problem illustrating POSIX shared-memory API.

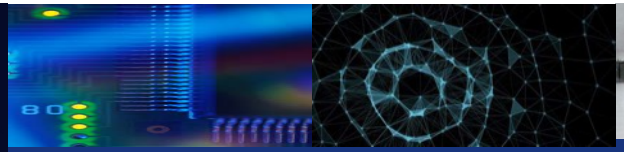
shmconsumer.c

```
/* gcc -lrt */
int main(int argc, char *argv[])
{
    int fd, shmsize, ret;
    void *shmptr;

    fd = shm_open(argv[1], O_RDONLY, 0444);
    if(fd == -1) {
        ERR_EXIT(“consumer: shm_open()");
    }

    shmsize = TEXT_NUM*sizeof(struct shared_struct);
    shmptr = (char *)mmap(0, shmsize, PROT_READ, MAP_SHARED, fd, 0);
    if(shmptr == (void *)-1) {
        ERR_EXIT(“consumer: mmap()");
    }

    printf("consumed message: %s\n", (char *)shmptr);
    return EXIT_SUCCESS;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>

#include "shmdata.h"
```

IPC-消息传递

- 进程之间相互通信，无需借助共享变量
- IPC设施提供两种操作：
  - Send 发送（消息）
  - Receive 接收（信息）
- 消息大小是固定的或可变的



IPC-消息传递

- 如果进程P和Q希望通信，则它们需要：
  - 在它们之间建立通信链接
  - 通过发送/接收交换消息
- 实施问题：
  - 如何建立联系？
  - 链接是否可以与两个以上的进程关联？
  - 每对通信进程之间可以有多少链路？
  - 链路的容量是多少？
  - 链接可以容纳的消息大小是固定的还是可变的？
  - 链接是单向的还是双向的？

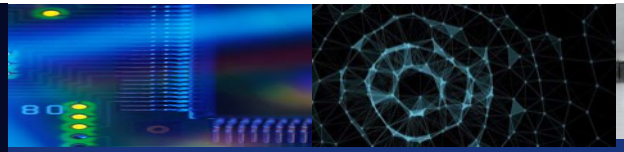
通信链路的实现

- 物理：
  - 共享内存
  - 硬件总线
  - 网络
- 逻辑：
  - 直接或间接
  - 同步还是异步
  - 自动或显式缓冲

直接通信

- 进程之间必须明确命名：
  - 发送（P，消息）－向进程P发送消息
  - 接收（Q，消息）－从流程Q接收消息
- 通信链路的特性：
  - 链接是自动建立的
  - 链路仅与一对通信进程相关联
  - 每对之间只存在一个链接
  - 链路可能是单向的，但通常是双向的

- 这种通信方式展示了寻址的对称性，即发送和接收进程必须指定对方，以便通信。
- 也可以采用寻址的非对称性，即只要发送者指定接收者，而接收者不需要指定发送者

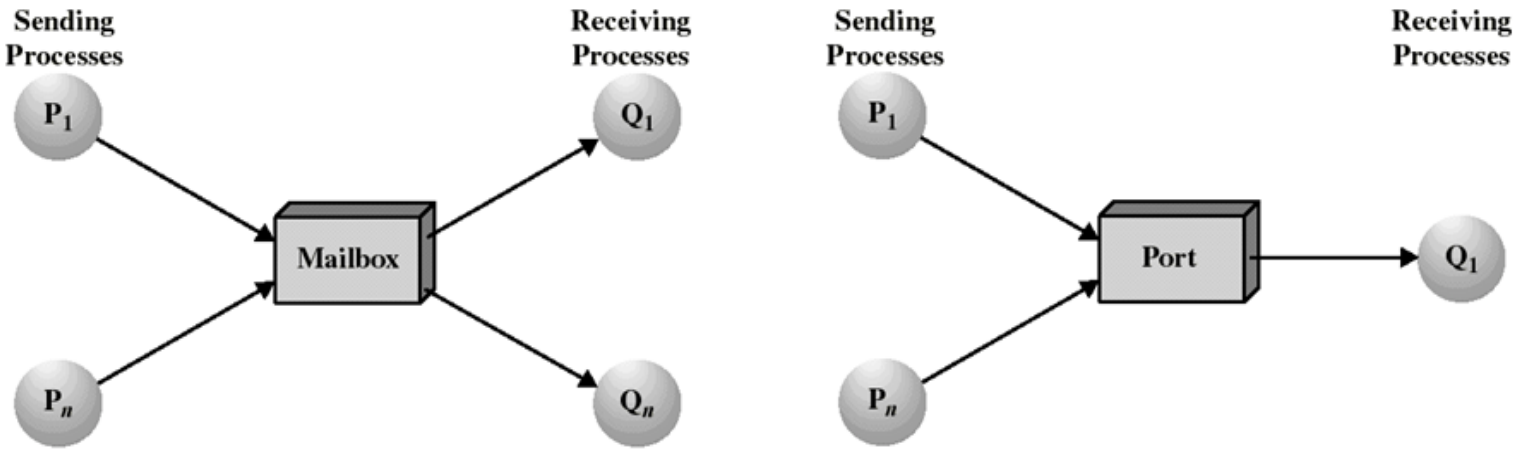



间接通信

- **通过邮箱或者端口来发送和接收消息**
  - 每个邮箱都有一个唯一的id
  - 进程只有在共享邮箱时才能通信
- **通信链路的特性**
  - 仅当进程共享公共邮箱时才建立链接
  - 链接可能与许多进程相关联
  - 每对进程可以共享多个通信链路
  - 链路可以是单向的或双向的

间接通信

- 操作
  - 创建新邮箱（端口）
  - 通过邮箱发送和接收邮件
  - 删除邮箱
- 基本体定义为：
  - 发送（A，消息）－将消息发送到邮箱A
  - 接收（A，消息）－从邮箱A接收消息



间接通信

- 邮箱共享
  - P1、P2和P3共享邮箱A
  - P1，发送；P2和P3接收
  - 谁得到消息？
- 解决
  - 允许链接最多与两个进程关联
  - 一次只允许一个进程执行接收操作
  - 允许系统任意选择接收器。通知发送方接收者是谁。
- 邮箱所有者

  邮箱可以被进程拥有；

  邮箱被操作系统拥有；

  邮箱可以转移；

同步

消息传递可以是阻塞的，也可以是非阻塞的

- **阻塞被认为是同步的**
  - 阻塞发送—在接收进程或者邮箱收到消息之前，发送进程阻塞；
  - Blocking receive（阻塞接收）——在消息可用之前，接收进程将被阻塞
- **非阻塞被认为是异步的**
  - 非阻塞发送——发送进程发送消息并继续执行；
  - 非阻塞接收——接收进程接收：
    - 有效消息，或
    - 空消息
- **可能有不同的组合**
  - 如果发送和接收都是阻塞的，双方之间有一个交会。

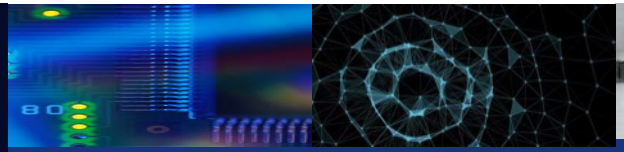
生产者-消费者（阻塞）消息传递

生产者

```
message next_produced;

while (true) {
     /* produce an item in next_produced */

     send(next_produced);
}
```

消费者

```
message next_consumed;

while (true) {
     receive(next_consumed);

     /* consume the item in next_consumed */
}
```



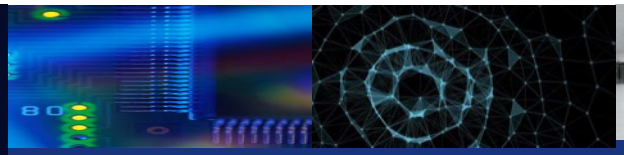
缓存

- 附加到链接的消息队列，不管通信是直接还是间接的；
- 以三种方式之一实现队列：

1. 零容量 - 链路上没有消息排队。发送方必须等待接收方

2. 有限容量 - 有限长度的n条消息，如果链路已满，则发送者必须等待

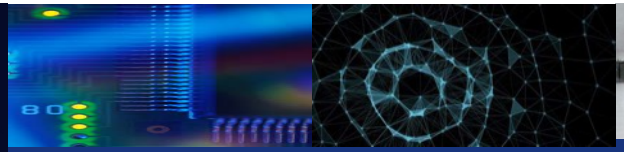
3. 无限容量 - 无限长，发送方从不等待



Linux: Message-passing

- Message structure

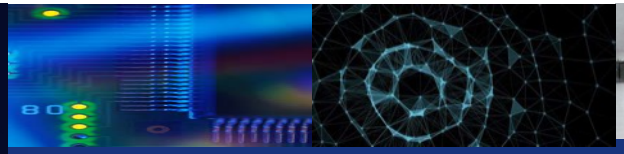
```
struct msg_st {
    long int msg_type;
    char mtext[TEXT_SIZE];
};

struct msqid_ds {
    struct ipc_perm msg_perm;
    time_t msg_stime;    /* Last msgsnd time */
    time_t msg_rtime;    /* Last msgrcv time */
    time_t msg_ctime;    /* Last change time */
    msgqnum_t msg_qnum;  /* Current number of messages in queue
*/
    msglen_t msg_qbytes; /* Maximum number of bytes allowed in
queue */
    pid_t msg_lspid;     /* PID of last msgsnd */
    pid_t msg_lrpid;     /* PID of last msgrcv */
```




Linux: Message-passing

- Functions

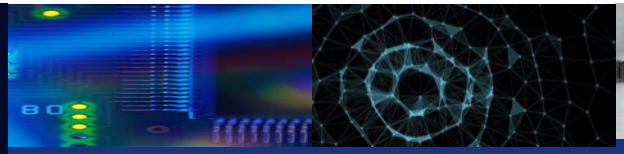
```
key_t ftok(char *pathname, char proj_id)

int msgget(key_t key, int msgflg)

int msgsnd(int msqid, struct msgbuf *msgp, int msgsz,
int msgflg)

int msgrcv(int msqid, struct msgbuf *msgp, int msgsz,
long msgtyp, int msgflg)

int msgctl(int msqid, int cmd, struct msqid_ds *buf)
```



Linux: Message-passing

- Sending & Receiving process illustrating Linux message-passing AP
  - msgdata.h

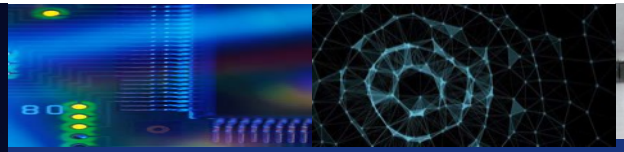
```
#define TEXT_SIZE 512
/* considering
------ Messages Limits --------
max queues system wide = 32000
max size of message (bytes) = 8192
default max size of queue (bytes) = 16384
----------------------------------------
The size of message is set to be 512, the total number of messages is 16384/512 = 32
If we take the max size 8192, the number would be 16384/8192 = 2. It is not reasonable
*/
/* message structure */
struct msg_struct {
    long int msg_type;
    char mtext[TEXT_SIZE]; /* binary data */
};
#define PERM S_IRUSR|S_IWUSR|IPC_CREAT
#define ERR_EXIT(m) \
    do { \
        perror(m); \
        exit(EXIT_FAILURE); \
    } while(0)
```



Linux: Message-passing msgsnd.c

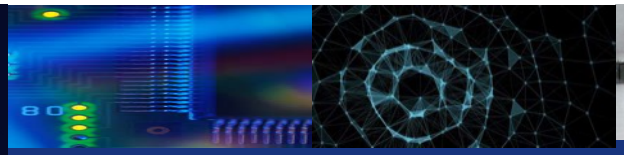
```
int main(int argc, char *argv[])
{
    struct msg_struct data;

    long int msg_type;
    char buffer[TEXT_SIZE], pathname[80];
    int msqid, ret, count = 0;
    key_t key;
    FILE *fp;
    struct stat fileattr;

    if(argc < 2) {
        printf("Usage: ./a.out pathname\n");
        return EXIT_FAILURE;
    }
    strcpy(pathname, argv[1]);

    if(stat(pathname, &fileattr) == -1) {
        ret = creat(pathname, O_RDWR);
        if (ret == -1) {
            ERR_EXIT("creat()");
        }
        printf("shared file object created\n");
    }
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/msg.h>
#include <sys/stat.h>
#include <fcntl.h>

#include “msgdata.h”
```



Linux: Message-passing msgsnd.c

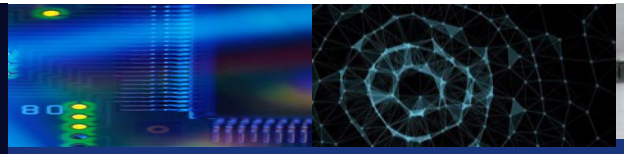
```
key = ftok(pathname, 0x27); /* project_id can be any nonzero integer */
    if(key < 0) {
        ERR_EXIT("ftok()");
    }

    printf("\nIPC key = 0x%x\n", key);

    msqid = msgget((key_t)key, 0666 | IPC_CREAT);
    if(msqid == -1) {
        ERR_EXIT("msgget()");
    }

    fp = fopen("./msgsnd.txt", "rb");
    if(!fp) {
        ERR_EXIT("source data file: ./msgsnd.txt fopen()");
    }

    struct msqid_ds msqattr;
    ret = msgctl(msqid, IPC_STAT, &msqattr);
    printf("number of messages remainded = %ld, empty slots = %ld\n",
msqattr.msg_qnum, 16384/TEXT_SIZE-msqattr.msg_qnum);
    printf("Blocking Sending ... \n");
```



Linux: Message-passing msgsnd.c

```
    while (!feof(fp)) {
            ret = fscanf(fp, "%ld %s", &msg_type, buffer);
            if (ret == EOF) break;
            printf("%ld %s\n", msg_type, buffer);

            data.msg_type = msg_type;
            strcpy(data.mtext, buffer);

            ret = msgsnd(msqid, (void *)&data, TEXT_SIZE, 0);
                /* 0: blocking send, waiting when msg queue is full */
            if(ret == -1) {
                ERR_EXIT("msgsnd()");
            }
            count++;
        }

        printf("number of sent messages = %d\n", count);

        fclose(fp);
        system("ipcs -q");
        exit(EXIT_SUCCESS);
    }
```

Demo

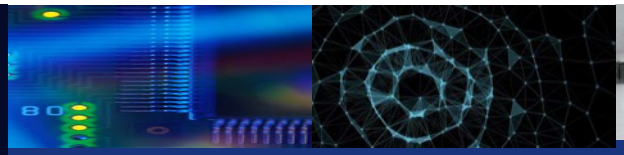


Linux: Message-passing

- Sending & Receiving process illustrating Linux message-passing API.
  - msgrcv.c (1)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/msg.h>
#include <sys/stat.h>

#include “msgdata.h”
```

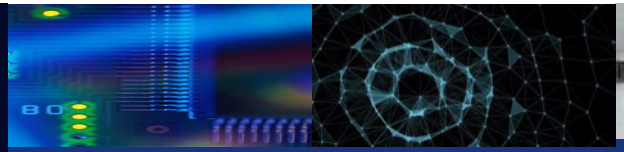
```
int main(int argc, char *argv[])
{
    key_t key;
    struct stat fileattr;
    char pathname[80];
    int msqid, ret, count = 0;
    struct msg_struct data;
    long int msgtype = 0;   /* 0 - type of any messages */

    if(argc < 2) {
        printf("Usage: ./msgrcv pathname msg_type\n");
        return EXIT_FAILURE;
    }
    strcpy(pathname, argv[1]);
    if(stat(pathname, &fileattr) == -1) {
        ERR_EXIT("shared file object stat error");
    }
    if((key = ftok(pathname, 0x27)) < 0) {
        ERR_EXIT("ftok()");
    }
    printf("\nIPC key = 0x%x\n", key);
```



Linux: Message-passing

msgrcv.c (2)

```
msqid = msgget((key_t)key, 0666); /* do not create a new msg queue */
    if(msqid == -1) {
        ERR_EXIT("msgget()");
    }

    if(argc < 3)
        msgtype = 0;
    else {
        msgtype = atol(argv[2]);
        if (msgtype < 0)
            msgtype = 0;
    }    /* determin msgtype (class number) */
    printf("Selected message type = %ld\n", msgtype);

    while (1) {
        ret = msgrcv(msqid, (void *)&data, TEXT_SIZE, msgtype,
IPC_NOWAIT);
            /* Non_blocking receive */
        if(ret == -1) { /* end of this msgtype */
            printf("number of received messages = %d\n", count);
            break;
        }

        printf("%ld %s\n", data.msg_type, data.mtext);
        count++;
    }
```



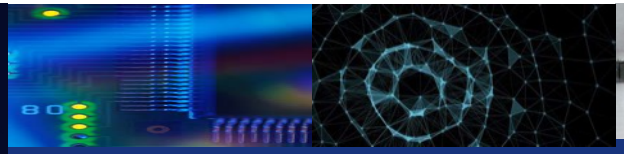
msgrcv.c (3)

```
struct msqid_ds msqattr;
   ret = msgctl(msqid, IPC_STAT, &msqattr);
   printf("number of messages remainding = %ld\n", msqattr.msg_qnum);

   if(msqattr.msg_qnum == 0) {
       printf("do you want to delete this msg queue?(y/n)");
       if (getchar() == 'y') {
           if(msgctl(msqid, IPC_RMID, 0) == -1)
               perror("msgctl(IPC_RMID)");
       }
   }

   system("ipcs -q");
   exit(EXIT_SUCCESS);
}
```

Linux: Message-passing

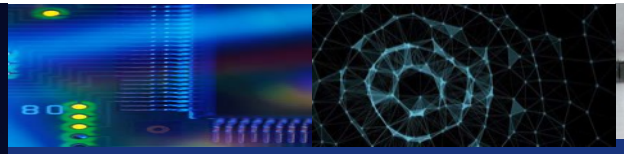
```
isscgy@ubuntu:/mnt/os-2020$ gcc -o b.out alg.9-2-msgrcv.c
isscgy@ubuntu:/mnt/os-2020$ ./b.out /home/isscgy/mysh
shared file object stat error: No such file or directory
isscgy@ubuntu:/mnt/os-2020$ ./b.out /home/isscgy/myshm 2

IPC key = 0x27011c6c
Selected message type = 2
2 Nami
2 Usopo
number of received messages = 2
number of messages remainding = 8

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x27011c9e 1          isscgy     666        0            0
0x27011c6c 5          isscgy     666        4096         8

isscgy@ubuntu:/mnt/os-2020$
```

运行结果



Linux: Message-passing

```
isscgy@ubuntu:/mnt/os-2020$ ./b.out /home/isscgy/myshm 0

IPC key = 0x27011c6c
Selected message type = 0
1 Luffy
1 Zoro
1 Sanji
3 Chopper
4 Robin
4 Franky
5 Brook
6 Sunny
number of received messages = 8
number of messages remainding = 0
do you want to delete this msg queue?(y/n)y

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x27011c9e 1          isscgy     666        0            0

isscgy@ubuntu:/mnt/os-2020$
```



POSIX: Message-passing

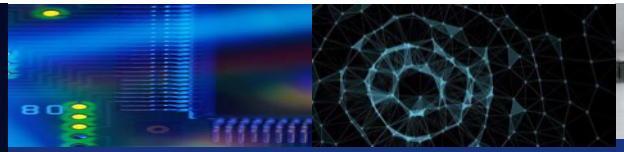
```
#include <mqueue.h>
```

- Open, Close and Unlink

```
mqd_t mq_open(const char *name, int oflag, mode_t mode, struct
mq_attr *attr ); /* return the mqdes, or -1 if failed */
mqd_t mqID;
mqID = mq_open("/anonymQueue", O_RDWR | O_CREAT, 0666,
NULL);

mqd_t mq_close(mqd_t mqdes);
mqd_t mq_unlink(const char *name); /* return -1 if failed */
```

- Send and Receive

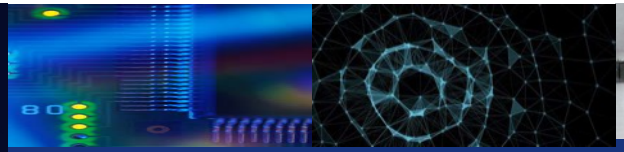
```
mqd_t mq_send(mqd_t mqdes, const char *msg_ptr, size_t msg_len,
unsigned msg_prio); /* return 0, or -1 if failed */
mq_send(mqID, msg, sizeof(msg), i)
mqd_t mq_receive(mqd_t mqdes, char *msg_ptr, size_t msg_len,
unsigned *msg_prio); /* return the number of char received, or -1
if failed */
mq_attr mqAttr;
mq_getattr(mqID, &mqAttr);
mq_receive(mqID, buf, mqAttr.mq_msgsize, NULL)
```



POSIX: Message-passing

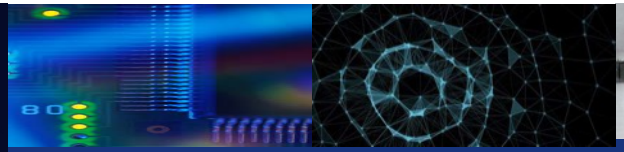
- Note that the Linux ipcs utility is not fully compatible to the POSIX ipcs utility. The message queues, shared memory and semaphores in POSIX can not be sensed by System V bash command such as

```
$ipcs -q
```



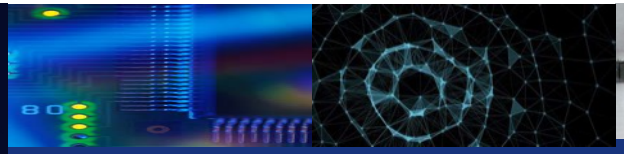

Mach

- Mach通信是基于消息的设计
  - 甚至系统调用也是消息
  - 每个任务在创建时获得两个端口-内核和通知
  - 使用mach_msg（）函数发送和接收消息
  - 通信所需的端口，通过创建mach端口分配（）
  - 收发灵活；例如，邮箱已满时有四个选项：
    - 无限期地等待
    - 最多等待n毫秒
    - 立即返回
    - 临时缓存消息



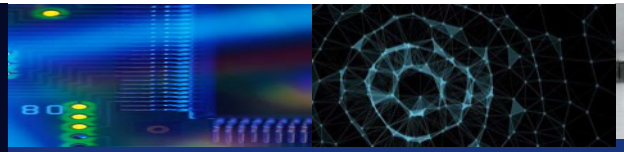
Mach消息结构

```
#include<mach/mach.h>

struct message {
   mach_msg_header_t header;
   int data;
};

mach_port_t client;
mach_port_t server;
```



Mach客户端

```
        /* Client Code */

struct message message;

// construct the header
message.header.msgh_size = sizeof(message);
message.header.msgh_remote_port = server;
message.header.msgh_local_port = client;

// send the message
mach_msg(&message.header, // message header
   MACH_SEND_MSG, // sending a message
   sizeof(message), // size of message sent
   0, // maximum size of received message - unnecessary
   MACH_PORT_NULL, // name of receive port - unnecessary
   MACH_MSG_TIMEOUT_NONE, // no time outs
   MACH_PORT_NULL // no notify port
);
```



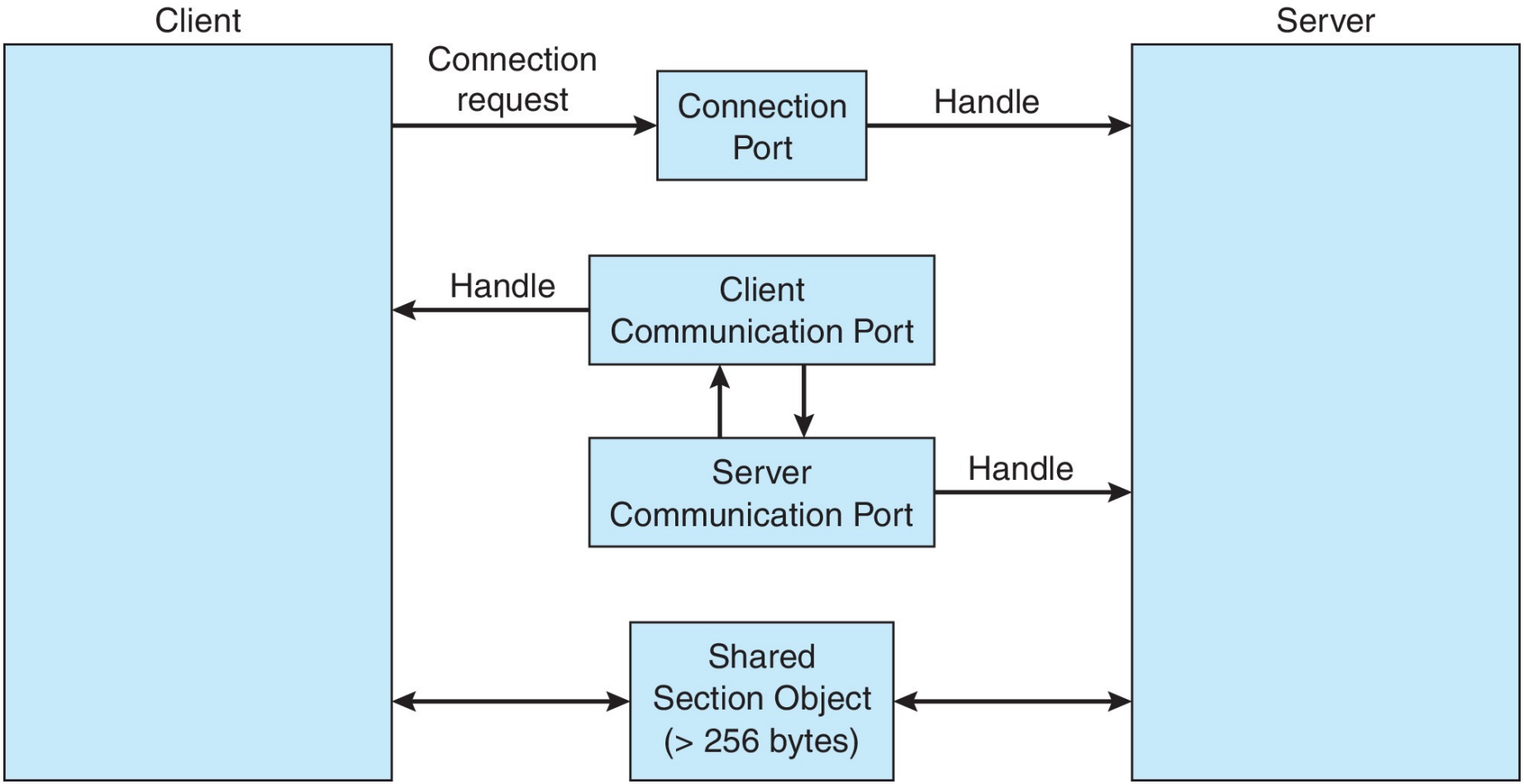
Mach服务器端

```
        /* Server Code */

struct message message;

// receive the message
mach_msg(&message.header, // message header
   MACH_RCV_MSG, // sending a message
   0, // size of message sent
   sizeof(message), // maximum size of received message
   server, // name of receive port
   MACH_MSG_TIMEOUT_NONE, // no time outs
   MACH_PORT_NULL // no notify port
);
```

IPC系统示例－Windows

- **通过高级本地过程调用（ALPC）设施以消息传递为中心**
  - 仅在同一系统上的两个进程之间工作，类似RPC；
  - 使用端口（如邮箱）建立和维护通信通道；
  - 使用两种类型的端口：连接端口和通信端口；
  - 通讯工作如下：
    - 客户端打开子系统**连接端口**对象的句柄；
    - 客户端发送一个连接请求；
    - 服务器创建一对专用通信端口，并将其中一个端口的句柄返回给客户端；
    - 客户端和服务器使用相应的端口句柄发送消息或回调，并监听回复；

Windows中的本地过程调用



Windows中的本地过程调用

When an ALPC channel is created, one of three message-passing techniques is chosen:

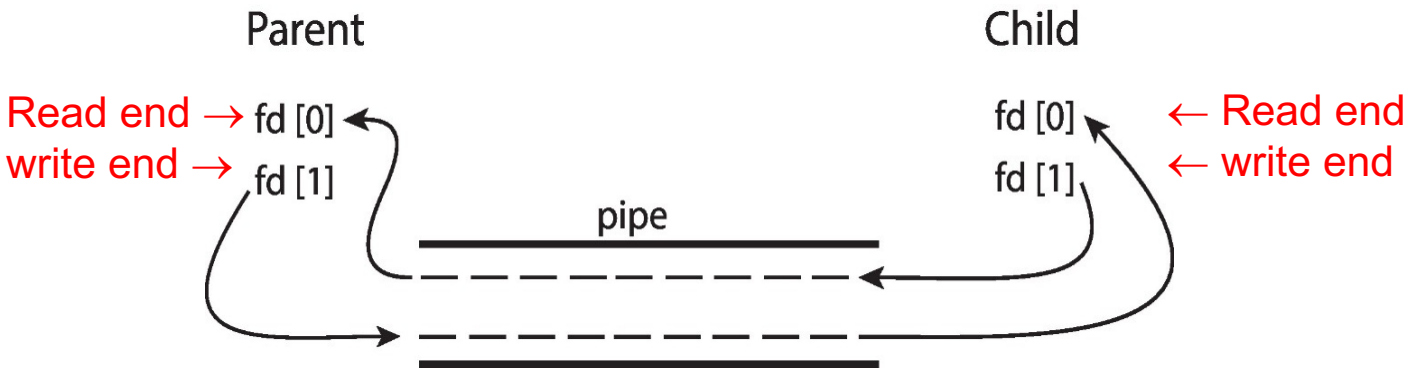
1. For small messages (up to 256 bytes), the port's message queue is used as intermediate storage, and the messages are copied from one process to the other.
2. Larger messages must be passed through a **section object**, which is a region of shared memory associated with the channel.
3. When the amount of data is too large to fit into a section object, an API is available that allows server processes to read and write directly into the address space of a client.

管道（PIPE）

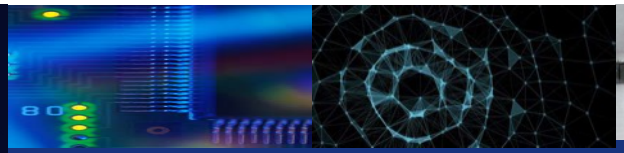
- 充当允许两个进程通信的管道
- 存在的问题：
  - 通信是单向的还是双向的？
  - 在双向通信的情况下，是半双工还是全双工？
  - 通信进程之间必须存在关系（即父子关系）？
  - 这些管道可以通过网络使用吗？还是在本地？
- 普通管道－无法从创建它的进程外部访问。通常，父进程创建管道并使用它与它创建的子进程通信。
- 命名管道－可以在没有父子关系的情况下访问。

普通管道（PIPE）

- 普通管道允许标准生产者-消费者风格的通信
- 生产者写入一端（管道的写入端）；
- 使用者从另一端（管道的读取端）读取数据
- 因此，普通管道是单向的；
- 需要通信进程之间的父子关系；



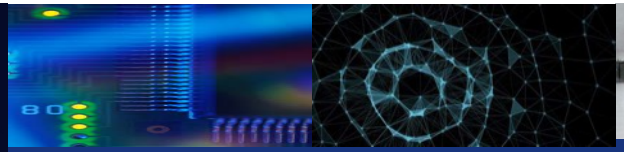
- Windows调用这些匿名管道



普通管道样例代码

```
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1

int main(void)
{
   char write_msg[BUFFER_SIZE] = "Greetings";
   char read_msg[BUFFER_SIZE];
   int fd[2];
   pid_t pid;

      /* Program continues in  Figure 3.22 */
```

```
/* create the pipe */
if (pipe(fd) == -1) {
   fprintf(stderr,"Pipe failed");
   return 1;
}

/* fork a child process */
pid = fork();

if (pid < 0) { /* error occurred */
   fprintf(stderr, "Fork Failed");
   return 1;
}

if (pid > 0) { /* parent process */
   /* close the unused end of the pipe */
   close(fd[READ_END]);

   /* write to the pipe */
   write(fd[WRITE_END], write_msg, strlen(write_msg)+1);

   /* close the write end of the pipe */
   close(fd[WRITE_END]);
}
else { /* child process */
   /* close the unused end of the pipe */
   close(fd[WRITE_END]);

   /* read from the pipe */
   read(fd[READ_END], read_msg, BUFFER_SIZE);
   printf("read %s",read_msg);

   /* close the read end of the pipe */
   close(fd[READ_END]);
}

return 0;
}
```

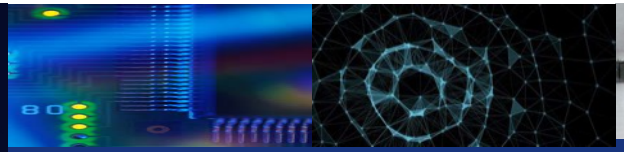


匿名管道

- 在window系统中普通的管道称为匿名管道；
- 与Unix的普通管道类似；
- 单向、父子进程之间传输；

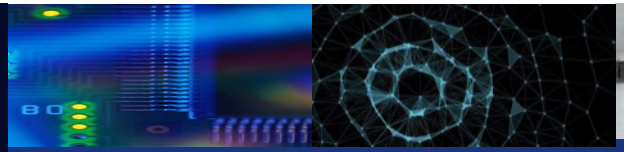
```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

#define BUFFER_SIZE 25

int main(VOID)
{
   HANDLE ReadHandle, WriteHandle;
   STARTUPINFO si;
   PROCESS_INFORMATION pi;
   char message[BUFFER_SIZE] = "Greetings";
   DWORD written;

      /* Program continues in  Figure 3.24 */
```

```
/* set up security attributes allowing pipes to be inherited */
SECURITY_ATTRIBUTES sa = {sizeof(SECURITY_ATTRIBUTES),NULL,TRUE};
/* allocate memory */
ZeroMemory(&pi, sizeof(pi));

/* create the pipe */
if (!CreatePipe(&ReadHandle, &WriteHandle, &sa, 0)) {
   fprintf(stderr, "Create Pipe Failed");
   return 1;
}

/* establish the START_INFO structure for the child process */
GetStartupInfo(&si);
si.hStdOutput = GetStdHandle(STD_OUTPUT_HANDLE);

/* redirect standard input to the read end of the pipe */
si.hStdInput = ReadHandle;
si.dwFlags = STARTF_USESTDHANDLES;

/* don't allow the child to inherit the write end of pipe */
SetHandleInformation(WriteHandle, HANDLE_FLAG_INHERIT, 0);

/* create the child process */
CreateProcess(NULL, "child.exe", NULL, NULL,
 TRUE, /* inherit handles */
 0, NULL, NULL, &si, &pi);

/* close the unused end of the pipe */
CloseHandle(ReadHandle);

/* the parent writes to the pipe */
if (!WriteFile(WriteHandle, message,BUFFER_SIZE,&written,NULL))
   fprintf(stderr, "Error writing to pipe.");

/* close the write end of the pipe */
CloseHandle(WriteHandle);

/* wait for the child to exit */
WaitForSingleObject(pi.hProcess, INFINITE);
CloseHandle(pi.hProcess);
CloseHandle(pi.hThread);
return 0;
}
```



匿名管道

```
#include <stdio.h>
#include <windows.h>

#define BUFFER_SIZE 25

int main(VOID)
{
HANDLE Readhandle;
CHAR buffer[BUFFER_SIZE];
DWORD read;

    /* get the read handle of the pipe */
    ReadHandle = GetStdHandle(STD_INPUT_HANDLE);

    /* the child reads from the pipe */
    if (ReadFile(ReadHandle, buffer, BUFFER_SIZE, &read, NULL))
       printf("child read %s",buffer);
    else
       fprintf(stderr, "Error reading from pipe");

    return 0;
}
```



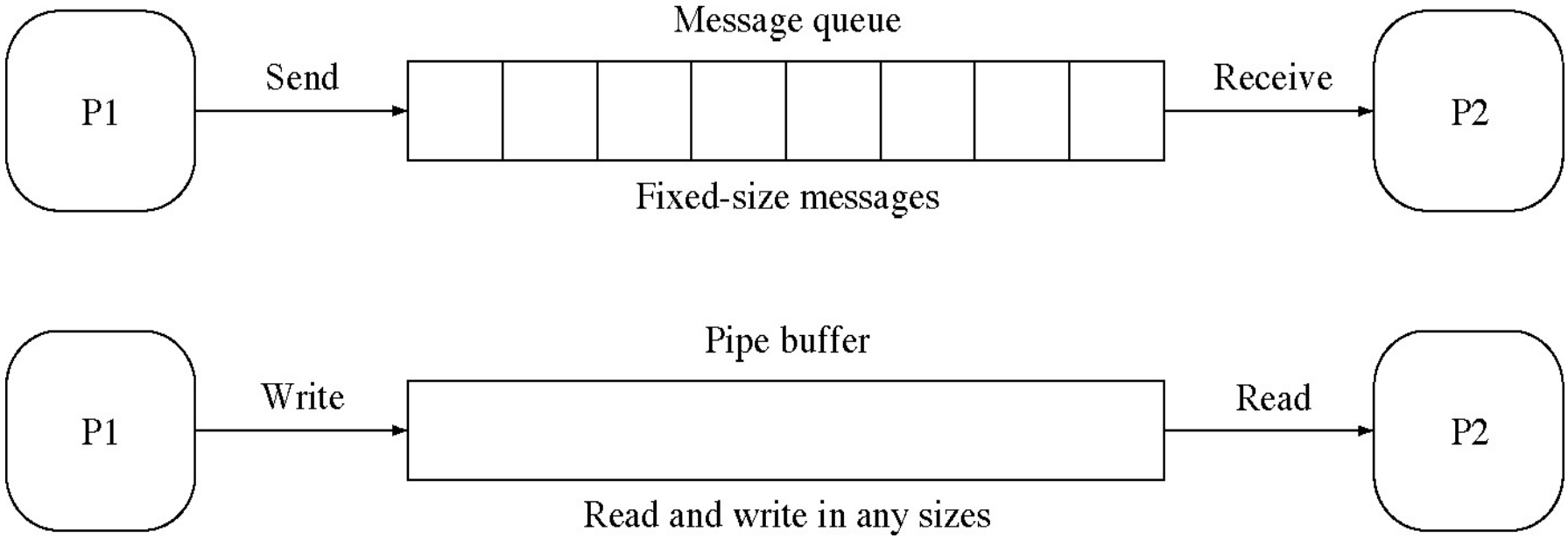
命名管道

- 命名管道比普通管道功能更强大
- 通信是双向的;
- 通信进程之间不需要父子关系;
- 有几个进程可以使用命名管道进行通信;
- 在UNIX和Windows系统上都提供;

Linux/Unix Pipe

- Named Pipes in UNIX/Linux
  - Named Pipes are referred to as FIFOs, created with the mkfifo() system call.
    - Once created, they appear as typical files in the file system and manipulated with the ordinary open(), read(), write(), and close() system calls.
  - A FIFO will continue to exist until it is explicitly deleted from the file system.
  - FIFOs allow bidirectional communication and half-duplex transmission.
    - If data must travel in both directions, two FIFOs are typically used.
  - The communicating processes must reside on the same machine/host.
    - use *sockets* if inter-machine communication is required

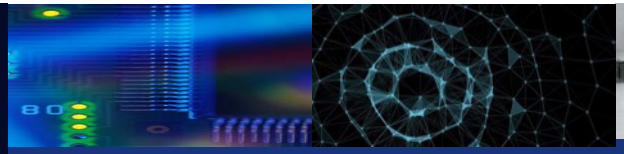
消息通信VS管道


Message queue
P1
Send
Receive
P2
Fixed-size messages
Pipe buffer
P1
Write
Read
P2
Read and write in any sizes

Linux: Pipes

- 普通管道

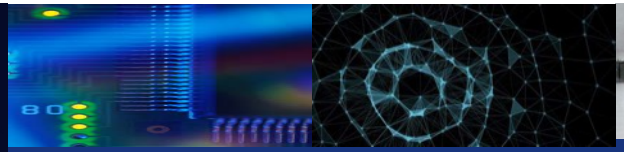
```
int pipefd[2];
int pipe(int pipefd);

int fcntl(int pipefd[0|1], int cmd);
int fcntl(int pipefd[0|1], int cmd, long
arg);

ssize_t write(int pipefd[1], void* buf,
size_t count);

ssize_t read(int pipefd[0], void* buf,
size_t count);

close(pipefd[0]);
close(pipefd[1]);
```



Linux: Pipes

- 命名管道

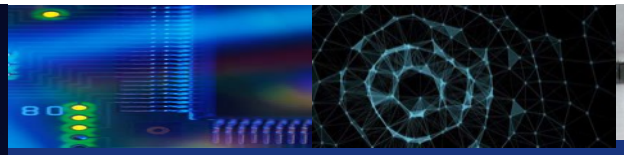
```
#define FIFO pathname /* pathname: "/tmp/my_fifo" */

unlink(FIFO); /*delete a name and possibly the file it refers
to */

mkfifo(FIFO, 0666);
int fdw = open(FIFO, O_RDWR);

mkfifo(FIFO, 0444);
int fdr = open(FIFO, O_RDONLY);

ssize_t write(int fdw, void* buf, size_t count);
    /* ssizt_t = signed int, sizt_t = unsigned int */

ssize_t read(int fdr, void* buf, size_t count);

close(fdw);
close(fdr);
```



Linux: Pipes

- 管道缓冲

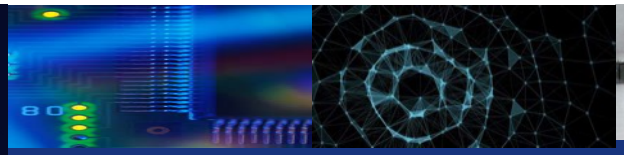
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#define ERR_EXIT(m) \
    do { \
        perror(m); \
        exit(EXIT_FAILURE); \
    } while (0)

int main(int argc, char *argv[])
{
    int pipefd[2];
    int bufsize;
    char *buffer;
    int flags, ret, lastwritten, count, totalwritten;

    if(pipe(pipefd) == -1) /* create an ordinary pipe */
        ERR_EXIT("pipe()");
    flags = fcntl(pipefd[1], F_GETFL);
    fcntl(pipefd[1], F_SETFL, flags | O_NONBLOCK); /* set
write_end NONBLOCK */
    bufsize = atoi(argv[1]);
    printf("testing buffer size = %d\n", bufsize);
    buffer = (char *)malloc(bufsize*sizeof(char));
    if(buffer == NULL || bufsize == 0)
        ERR_EXIT("malloc()");
```



Linux: Pipes

- 管道缓冲

```
 count = 0;
    while (1) {
        ret = write(pipefd[1], buffer,
bufsize);
            /* bufsize is better to be 2^k
*/
        if(ret == -1) {
            perror("write()");
            break;
        }
        lastwritten = ret;
        count++;
    }
    totalwritten = (count-1)*bufsize +
lastwritten;
    printf("single pipe buffer count =  %d,
last written = %d bytes\n", count,
lastwritten);
    printf("total written = %d bytes = %d
KiB\n", totalwritten, totalwritten/1024); /*
pipe buffer */

    return 0;
}
```

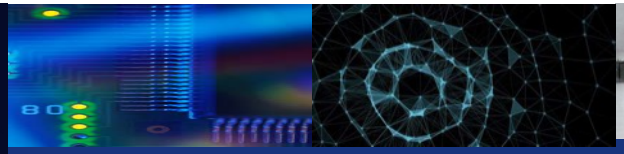


Linux: Pipes

- 管道缓冲

```
isscgy@ubuntu:/mnt/os-2020$ ./a.out 1024
testing buffer size = 1024
write(): Resource temporarily unavailable
single pipe buffer count =  64, last written = 1024 bytes
total written = 65536 bytes = 64 KiB
isscgy@ubuntu:/mnt/os-2020$ ./a.out 4096
testing buffer size = 4096
write(): Resource temporarily unavailable
single pipe buffer count =  16, last written = 4096 bytes
total written = 65536 bytes = 64 KiB
isscgy@ubuntu:/mnt/os-2020$ ./a.out 4097
testing buffer size = 4097
write(): Resource temporarily unavailable
single pipe buffer count =  11, last written = 4096 bytes
total written = 45066 bytes = 44 KiB
isscgy@ubuntu:/mnt/os-2020$ ./a.out 32768
testing buffer size = 3276
write(): Resource temporar
single pipe buffer count =
total written = 65536 byte
isscgy@ubuntu:/mnt/os-2020
testing buffer size = 3276
write(): Resource temporar
single pipe buffer count =
total written = 61442 byte
isscgy@ubuntu:/mnt/os-2020
```

Differs from PIPE_BUF in that PIPE_SIZE is the length of the actual memory allocation, whereas PIPE_BUF makes atomicity guarantees.
check the header of include/linux/pipi_fs_i.h
#define PIPE_SIZE PAGE_SIZE /* 4 KiB */
#define PIPE_DEF_BUFFERS 16 /* 4*16 = 64 KiB */



Linux: Pipes

- 管道容量

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#define PIPE_SIZE 64*1024 /* 64 KiB */
```

```
int main(void)
{
    char buf[PIPE_SIZE];
    int testfd[600][2];
    int i;
    long int ret;

    for (i = 0; i < 600; i++) {
        if(pipe(testfd[i]) == -1) {
            perror("pipe()");
            break;
        }
        fcntl(testfd[i][1], F_SETFL, O_NONBLOCK);
        ret = write(testfd[i][1], buf, PIPE_SIZE);
        if(ret == -1 || ret != PIPE_SIZE) {
            perror("write()");
            break;
        }
    }
    printf("\nsingle pipe buffer = 64 KiB, pipes created: %d\n", i);
    printf("total used size: %ld bytes = %ld KiB, or %.0f MiB\n", (long
int)i*PIPE_SIZE, (long int)i*PIPE_SIZE/1024,
(double)i*PIPE_SIZE/1024/1024);
    return 0;
}
```

Linux: Pipes

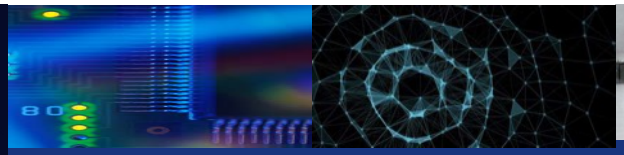
- 管道容量

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#define PIPE_SIZE 64*1024 /* 64 KiB */
```

```
int main(void)
{
    char buf[PIPE_SIZE];
    int testfd[600][2];
    int i;
    long int ret;

    for (i = 0; i < 600; i++) {
        if(pipe(testfd[i]) == -1) {
```

```
isscgy@ubuntu:/mnt/os-2020$ gcc alg.10-2-pipecapacity.c
isscgy@ubuntu:/mnt/os-2020$ ./a.out
pipe(): Too many open files

single pipe buffer = 64 KiB, pipes created: 510
total used size: 33423360 bytes = 32640 KiB, or 32 MiB
isscgy@ubuntu:/mnt/os-2020$
```

```
        }
    }
    printf("\nsingle pipe buffer = 64 KiB, pipes created: %d\n", i);
    printf("total used size: %ld bytes = %ld KiB, or %.0f MiB\n", (long
int)i*PIPE_SIZE, (long int)i*PIPE_SIZE/1024,
(double)i*PIPE_SIZE/1024/1024);
    return 0;
}
```



Linux: Pipes

➢ 普通管道

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#define READ_END 0
#define WRITE_END 1
```

```
/* a blocking read version */
int main(void)
{
    char write_msg[BUFSIZ]; /* BUFSIZ = 8192bytes, saved from stdin */
    char read_msg[BUFSIZ];
    int pipefd[2]; /* pipefd[0] for READ_END, pipefd[1] for WRITE_END */
    int flags;
    pid_t pid;

    if(pipe(pipefd) == -1) { /* create a pipe */
        perror("pipe()");
        exit(EXIT_FAILURE);
    }
    flags = fcntl(pipefd[WRITE_END], F_GETFL);
    fcntl(pipefd[WRITE_END], F_SETFL, flags | O_NONBLOCK); /* non-blocking
write */
    flags = fcntl(pipefd[READ_END], F_GETFL);
    fcntl(pipefd[READ_END], F_SETFL, flags); /* blocking read */

    pid = fork(); /* fork a child process */
    if(pid < 0) {
        perror("fork()");
        exit(EXIT_FAILURE);
```

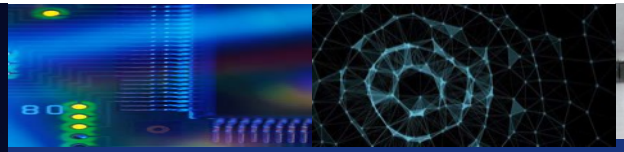
Linux: Pipes

- 普通管道

```
if(pid > 0) { /* parent process */
        while (1) {
            printf("Enter some text: ");
            fgets(write_msg, BUFSIZ, stdin);
```

```
isscgy@ubuntu:/mnt/os-2020$ gcc alg.10-3-pipe-ord-1.c
isscgy@ubuntu:/mnt/os-2020$ ./a.out
Enter some text: hello world

                                        pipe read = hello world
Enter some text: good morning

                                        pipe read = good morning
Enter some text: end

                                        pipe read = end
isscgy@ubuntu:/mnt/os-2020$
```

```
            wait(0);
            close(pipefd[WRITE_END]);
            close(pipefd[READ_END]);
            exit(EXIT_SUCCESS);
        }
```




Linux: Pipes

- Pipes in UNIX CLI
  - A pipe can be constructed on the UNIX command line using the | character. The complete command is

    ls | less

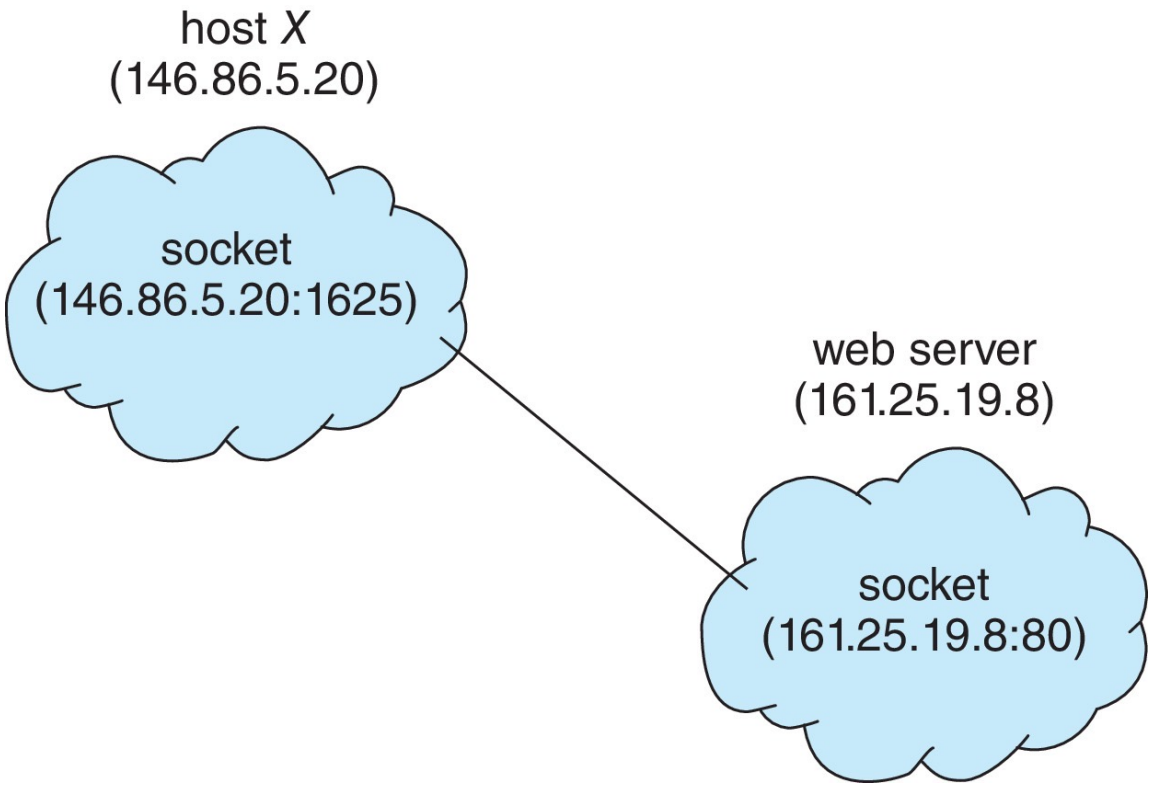
  - The commands ls and less are running as individual processes. The output of ls is delivered as the input to less. Result of ls:

客户机-服务器系统中的通信

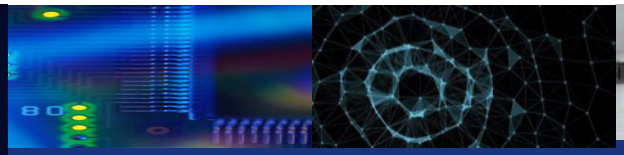
- 套接字（Socket）
- 远程过程调用（RPC）
- 套接字被定义为通信的端点
- IP地址和端口的串联－包含在消息包开头的数字，用于区分主机上的网络服务；
- 套接字161.25.19.8:1625引用主机161.25.19.8上的端口1625；
- 通信由一对套接字组成
- 1024以下的所有端口都是相对固定的，用于标准服务；
- 特殊IP地址127.0.0.1（环回），用于表示正在运行进程的系统

Demo

套接字


host X
(146.86.5.20)
socket
(146.86.5.20:1625)
web server
(161.25.19.8)
socket
(161.25.19.8:80)

Demo



套接字

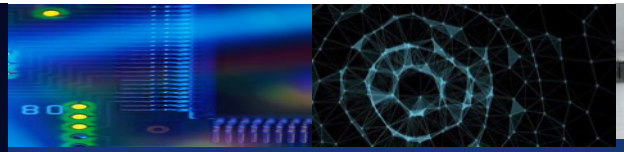
- **三种类型的套接字**
  - 面向连接（TCP）
  - 无连接（UDP）
  - MulticastSocket类–数据可以发送到多个收件人
- **考虑java中的这个"日期"服务器：**

```
import java.net.*;
import java.io.*;

public class DateServer
{
   public static void main(String[] args) {
      try {
         ServerSocket sock = new ServerSocket(6013);

         /* now listen for connections */
         while (true) {
            Socket client = sock.accept();

            PrintWriter pout = new
             PrintWriter(client.getOutputStream(), true);

            /* write the Date to the socket */
            pout.println(new java.util.Date().toString());

            /* close the socket and resume */
            /* listening for connections */
            client.close();
         }
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```

套接字

客户机的等效日期

```
import java.net.*;
import java.io.*;

public class DateClient
{
   public static void main(String[] args) {
      try {
         /* make connection to server socket */
         Socket sock = new Socket("127.0.0.1",6013);

         InputStream in = sock.getInputStream();
         BufferedReader bin = new
            BufferedReader(new InputStreamReader(in));

         /* read the date from the socket */
         String line;
         while ( (line = bin.readLine()) != null)
            System.out.println(line);

         /* close the socket connection*/
         sock.close();
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```



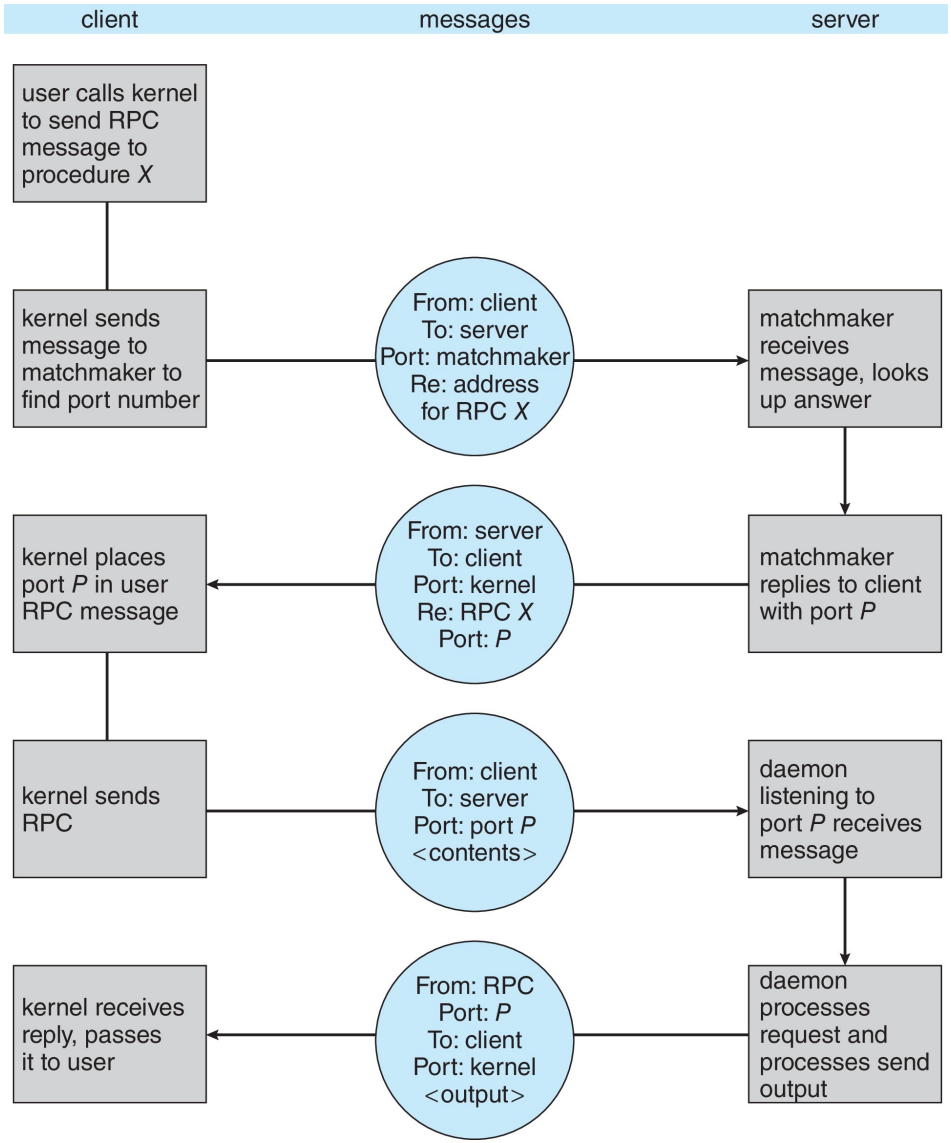
远程过程调用

- 远程过程调用（RPC）抽象网络系统上进程之间的过程调用
  - 再次使用端口进行服务区分
- Stubs－服务器上实际过程的客户端代理
- 客户端存根定位服务器并封送参数
- 服务器端存根接收此消息，解压缩封送的参数，并在服务器上执行该过程
- 在Windows上，存根代码根据用Microsoft接口定义语言（MIDL）编写的规范编译

远程过程调用

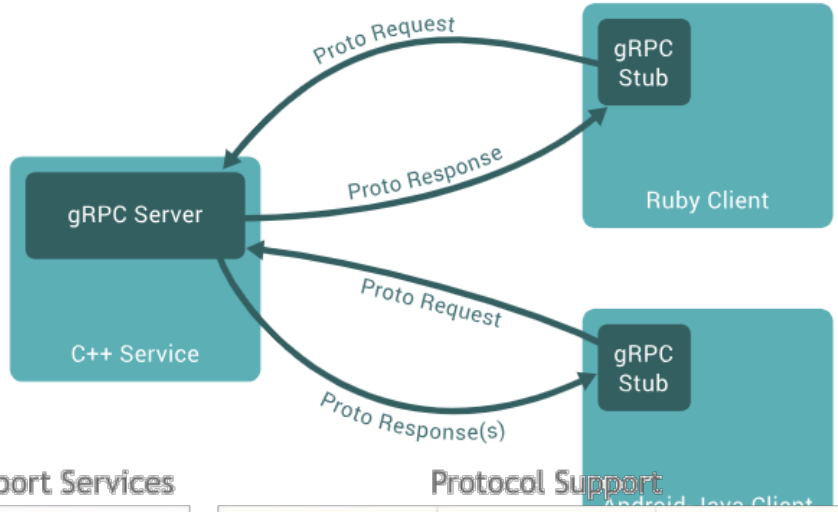
- 通过外部数据表示（XDL）格式处理数据表示，以考虑不同的体系结构
  - 大端和小端;
- 远程通信比本地通信有更多的故障场景
  - 消息可以只传递一次，而不是最多一次
- 操作系统通常提供会合（或配对）服务来连接客户端和服务器

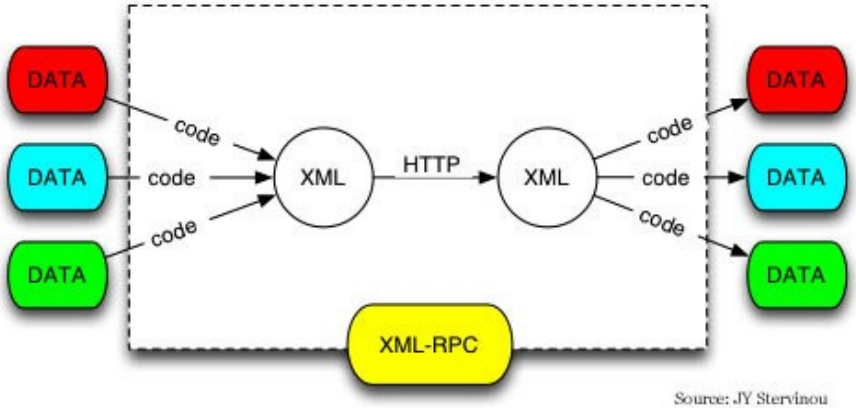
远程过程调用


client
messages
server
user calls kernel to send RPC message to procedure X
kernel sends message to matchmaker to find port number
From: client To: server Port: matchmaker Re: address for RPC X
matchmaker receives message, looks up answer
kernel places port P in user RPC message
From: server To: client Port: kernel Re: RPC X Port: P
matchmaker replies to client with port P
kernel sends RPC
From: client To: server Port: port P <contents>
daemon listening to port P receives message
kernel receives reply, passes it to user
From: RPC Port: P To: client Port: kernel <output>
daemon processes request and processes send output

RPC的实际应用

- **关键点**
  - 对象序列化协议；
  - 调用控制协议；





Source: JY Stervinou

Transport Services

Socket & Datagram
HTTP Tunnel
In-VM Pipe

Protocol Support

HTTP & WebSocket	SSL · StartTLS	Google Protobuf
zlib/gzip Compression	Large File Transfer	RTSP
Legacy Text · Binary Protocols with Unit Testability		

Apache Thrift ™ Core

Extensible Event Model
Universal Communication API
Zero-Copy-Capable Rich Byte Buffer

DCE RPC

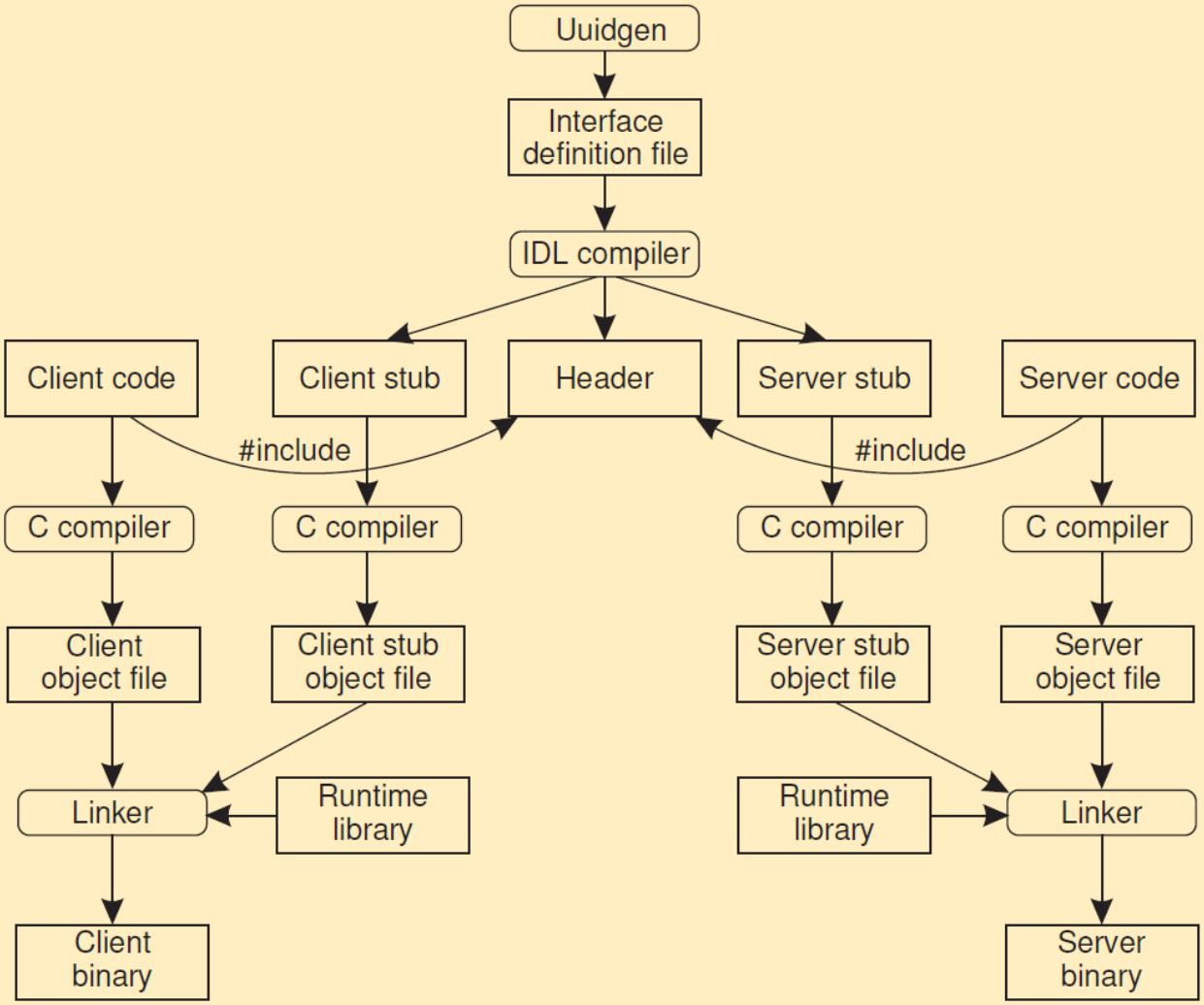
https://en.wikipedia.org/wiki/Distributed_Computing_Environment

- **DCE（分布式计算环境）**

  OSF（开放软件基金会）下的一个软件系统框架（慢慢消亡）

- **DCE的构成**
  - 远程过程调用 DCE/RPC；
  - 命名/目录服务；
  - 时间服务；
  - 安全服务；
  - 分布式文件系统 DCE/DFS；

DCE RPC


Uuidgen
Interface definition file
IDL compiler
Client code
Client stub
Header
Server stub
Server code
#include
#include
C compiler
C compiler
C compiler
C compiler
Client object file
Client stub object file
Server stub object file
Server object file
Linker
Runtime library
Runtime library
Linker
Client binary
Server binary

RPC实现过程

- **Linux的RPC编译工具：rpcgen**
- **先编写接口文件：rpc_test.x**

```
program TESTPROG {
version VERSION {
    string TEST(string) = 1;
} = 1;
} = 1;
```

版本编号

RPC程序编号

RPC函数

- **编译接口文件：rpcgen rpc_test.x**

  产生 rpc_test_clnt.c、 rpc_test_svc.c、rpc_test.h

RPC实现过程

- **rpc_test_clnt.c的内容**

```
/*
 * Please do not edit this file.
 * It was generated using rpcgen.
 */

#include <memory.h> /* for memset */
#include "rpc_test.h"

/* Default timeout can be changed using clnt_control() */
static struct timeval TIMEOUT = { 25, 0 };

char **
test_1(char **argp, CLIENT *clnt)
{
        static char *clnt_res;

        memset((char *)&clnt_res, 0, sizeof(clnt_res));
        if (clnt_call (clnt, TEST,
                (xdrproc_t) xdr_wrapstring, (caddr_t) argp,
                (xdrproc_t) xdr_wrapstring, (caddr_t) &clnt_res,
                TIMEOUT) != RPC_SUCCESS) {
                return (NULL);
        }
        return (&clnt_res);
}
~
~
```

RPC实现过程

rpc_test.h

```
/*
 * Please do not edit this file.
 * It was generated using rpcgen.
 */

#ifndef _RPC_TEST_H_RPCGEN
#define _RPC_TEST_H_RPCGEN

#include <rpc/rpc.h>


#ifdef __cplusplus
extern "C" {
#endif


#define TESTPROG 87654321
#define VERSION 1

#if defined(__STDC__) || defined(__cplusplus)
#define TEST 1
extern  char ** test_1(char **, CLIENT *);
extern  char ** test_1_svc(char **, struct svc_req *);
extern int testprog_1_freeresult (SVCXPRT *, xdrproc_t, caddr_t);

#else /* K&R C */
#define TEST 1
extern  char ** test_1();
extern  char ** test_1_svc();
extern int testprog_1_freeresult ();
#endif /* K&R C */

#ifdef __cplusplus
}
#endif

#endif /* !_RPC_TEST_H_RPCGEN */
```

RPC实现过程

rpc_test_svc.c

```
/*
 * Please do not edit this file.
 * It was generated using rpcgen.
 */

#include "rpc_test.h"
#include <stdio.h>
#include <stdlib.h>
#include <rpc/pmap_clnt.h>
#include <string.h>
#include <memory.h>
#include <sys/socket.h>
#include <netinet/in.h>

#ifndef SIG_PF
#define SIG_PF void(*)(int)
#endif

static void
testprog_1(struct svc_req *rqstp, register SVCXPRT *transp)
{
        union {
                char *test_1_arg;
        } argument;
        char *result;
        xdrproc_t _xdr_argument, _xdr_result;
        char *(*local)(char *, struct svc_req *);

        switch (rqstp->rq_proc) {
        case NULLPROC:
                (void) svc_sendreply (transp, (xdrproc_t) xdr_void, (char *)NULL);
                return;

        case TEST:
                _xdr_argument = (xdrproc_t) xdr_wrapstring;
                _xdr_result = (xdrproc_t) xdr_wrapstring;
                local = (char *(*)(char *, struct svc_req *)) test_1_svc;
                break;
```

RPC实现过程

- **生成client stub代码：rpcgen –Sc –o rpc_client.c rpc_test.x**

```
/*
 * This is sample code generated by rpcgen.
 * These are only templates and you can use them
 * as a guideline for developing your own functions.
 */

#include "rpc_test.h"


void
testprog_1(char *host)
{
        CLIENT *clnt;
        char * *result_1;
        char * test_1_arg;

#ifndef DEBUG
        clnt = clnt_create (host, TESTPROG, VERSION, "udp");
        if (clnt == NULL) {
                clnt_pcreateerror (host);
                exit (1);
        }
#endif  /* DEBUG */

        result_1 = test_1(&test_1_arg, clnt);
        if (result_1 == (char **) NULL) {
                clnt_perror (clnt, "call failed");
        }
#ifndef DEBUG
        clnt_destroy (clnt);
#endif   /* DEBUG */
}


int
main (int argc, char *argv[])
{
        char *host;

        if (argc < 2) {
                printf ("usage: %s server_host\n", argv[0]);
                exit (1);
```

RPC实现过程

- **生成server stub代码：rpcgen –Ss –o rpc_server.c rpc_test.x**

```
/*
 * This is sample code generated by rpcgen.
 * These are only templates and you can use them
 * as a guideline for developing your own functions.
 */

#include "rpc_test.h"

char **
test_1_svc(char **argp, struct svc_req *rqstp)
{
        static char * result;

        /*
         * insert server code here
         */

        return &result;
}
~
```

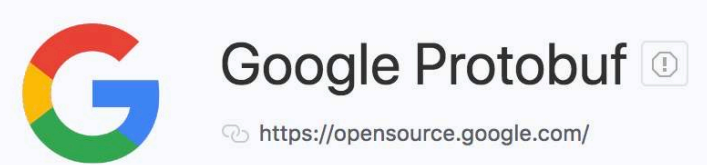
RPC实现过程

```
#include <time.h>
#include "test.h"

char ** test_1_svc(char **argp, struct svc_req *rqstp)
{
static char * result;
static char tmp_char[128];
time_t rawtime;

/*
* insert server code here
*/
if( time(&rawtime) == ((time_t)-1) ) {
strcpy(tmp_char, "Error");
result = tmp_char;
return &result;
}
sprintf(tmp_char, "服务器当前时间是 :%s", ctime(&rawtime));
result = tmp_char;

return &result;
}
```

1. **gcc -Wall -o rpc_client rpc_client.c rpc_test_clnt.c**
2. **gcc -Wall -o rpc_server rpc_server.c rpc_test_svc.c**

服务器端代码

Protobuf & gRPC

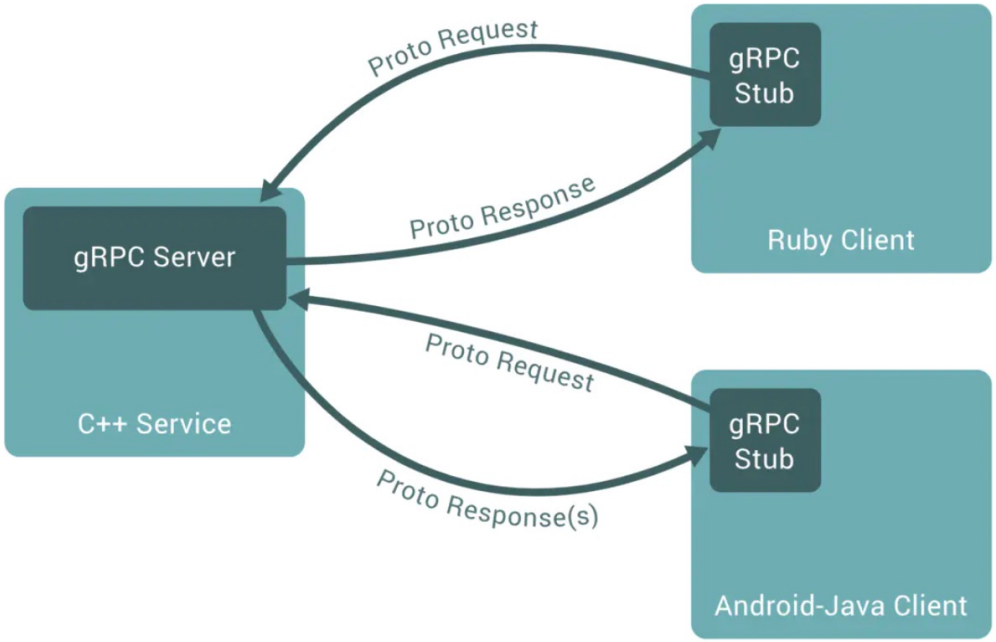


- **语言无关、平台无关:** 即 ProtoBuf 支持 Java、C++、Python 等多种语言，支持多个平台
- **高效:** 即比 XML 更小（3 ~ 10倍）、更快（20 ~ 100倍）、更为简单;
- **扩展性、兼容性好:** 你可以更新数据结构，而不影响和破坏原有的旧程序;

```
message Example1 {
    optional string stringVal = 1;
    optional bytes bytesVal = 2;
    message EmbeddedMessage {
        int32 int32Val = 1;
        string stringVal = 2;
    }
    optional EmbeddedMessage embeddedExample1 = 3;
    repeated int32 repeatedInt32Val = 4;
    repeated string repeatedStringVal = 5;
}
```

Protobuf & gRPC

gRPC和restful API都提供了一套通信机制，用于server/client模型通信，而且它们都使用http作为底层的传输协议。



- gRPC可以通过protobuf来定义接口，从而可以有更加严格的接口约束条件；
- 通过protobuf可以将数据序列化为二进制编码，这会大幅减少需要传输的数据量；
- gRPC可以方便地支持流式通信；

Demo

谢谢